

پروژه درس شناسایی الگو

نام دانشجو:

حوریا مهری

نام استاد:

دکتر شالباف

خرداد 1405

فهرست

1	روش کار
7	بهبود تصویر
7	قسمت a
16	قسمت b
21	قسمت c
27	قسمت d
33	قسمت e
38	قسمت f
43	قسمت g
47	استخراج ویژگی
59	کلاس بندی
59	قسمت a
65	قسمت b
73	قسمت c
88	قسمت d و e

روش کار پروژه شناسایی الگو در تصاویر اولتراسوند کبد

محیط توسعه و ابزارها

کلیه مراحل پردازش تصویر، استخراج ویژگی، داده‌افزایی و پیاده‌سازی مدل‌های یادگیری ماشین و یادگیری عمیق در محیط Anaconda و با زبان برنامه‌نویسی Python انجام شده است.

در این پروژه از ابزارها و کتابخانه‌های زیر استفاده شده است:

- NumPy برای محاسبات ماتریسی و مدیریت داده‌های عددی
- OpenCV برای عملیات پیش‌پردازش، بهبود تصویر، اعمال فیلترهای فضایی - فرکانسی و استخراج لبه‌ها
- Matplotlib و Seaborn برای رسم نمودارها، هیستوگرام‌ها و تحلیل‌های بصری
- Pyradiomics برای استخراج ویژگی‌های رادیومیکس مرتبه اول و دوم
- TensorFlow و Keras برای پیاده‌سازی مدل EfficientNetB1، داده‌افزایی، آموزش و ارزیابی مدل
- Scikit-learn برای پیاده‌سازی مدل‌های یادگیری ماشین، نرمال‌سازی، تقسیم داده و محاسبه معیارهای عملکرد

بیان مسئله

هدف این پژوهش، توسعه یک سامانه مبتنی بر پردازش تصویر و شناسایی الگو جهت طبقه‌بندی میزان تجمع چربی کبد در تصاویر سونوگرافی است.

مطابق ساختار داده و مقاله مرجع، تصاویر به سه کلاس تقسیم شدند:

- کلاس 0: چربی کمتر از 33٪
- کلاس 1: چربی بین 33 تا 66٪

- **کلاس 2:** چربی بیشتر از 67%

فرآیند کامل پروژه شامل سه فاز اصلی است:

1. بهبود و پیش‌پردازش تصاویر
2. استخراج ویژگی‌های بافتی
3. کلاس‌بندی با روش‌های یادگیری ماشین و یادگیری عمیق

در ادامه مراحل کار به صورت دقیق تشریح می‌گردد.

1. پیش‌پردازش و بهبود تصویر

به دلیل وجود نویز speckle، کنتراست پایین، و عدم وضوح مرزهای کبد و کلیه راست، مجموعه‌ای از روش‌های بهبود تصویر اعمال شد. هدف این بخش افزایش کیفیت بصری و استانداردسازی تصاویر پیش از استخراج ویژگی‌ها بود.

1-1. هیستوگرام و متعادل‌سازی روشنایی

برای تحلیل توزیع شدت روشنایی و افزایش کنتراست:

- هیستوگرام هر تصویر استخراج شد.
- Global Histogram Equalization برای بهبود کلی کنتراست اعمال شد.
- statistical local enhancement مبتنی بر ارزیابی آماری برای بهبود محلی کنتراست اعمال شد
- CLAHE (Local Enhancement) برای تقویت کنتراست در نواحی بافتی و آشکارسازی پرده شکمی استفاده گردید.

1-2. فیلترهای فضایی (Spatial Filters)

با هدف افزایش وضوح و آشکارسازی مرزها، فیلترهای مشتق اعمال شدند:

- Sobel برای استخراج تغییرات مرتبه اول

- Laplacian در انواع مختلف برای استخراج لبه‌های مرتبه دوم
- تحلیل تأثیر هر ماسک لاپلاسیان روی نمایش مرز کبد انجام شد.

1-3. فیلترهای حوزه فرکانس

برای حذف نویزهای دانه‌ای و حفظ ساختار بافتی به کار گرفته شدند و تأثیر آن‌ها ارزیابی شد.

1-4. بررسی رفتار تصویر تحت نویز

برای درک بهتر الگوهای شدت و نحوه عملکرد فیلترها:

- Salt & Pepper
- Gaussian Noise
- Exponential Noise

به تصاویر افزوده شد و هیستوگرام هر تصویر قبل و بعد از نویز بررسی شد.

1-5. انتخاب بهترین فیلترهای بهبود تصویر

با تحلیل نتایج، چهار فیلتر به عنوان بهترین فیلترها انتخاب و با تصویر اصلی مقایسه شدند.

1-6. پردازش مورفولوژیک

برای جداسازی تقریبی نواحی کبد و ساختارهای اطراف، عملیات:

- Erosion
- Dilation
- Opening
- Closing

اعمال شد و خطوط مرزی کلیه و کبد تا حد امکان استخراج گردید.

2. استخراج ویژگی (Feature Extraction)

پس از استانداردسازی تصاویر، دو گروه ویژگی استخراج شد:

2-1. ویژگی‌های رادیومیکس

به کمک Pyradiomics و با اعمال ماسک مناسب:

- ویژگی‌های مرتبه اول: میانگین، واریانس، skewness، kurtosis

- ویژگی‌های بافت مرتبه دوم:

GLCM (در زوایای 0، 45 و 90 درجه) با همسایگی‌های 1، 2، و 3

تمام ویژگی‌ها برای جلوگیری از بایاس، در بازه [0,1] نرمال‌سازی شده و در یک فایل CSV ذخیره گردیدند.

2-2. ویژگی‌های یادگیری عمیق

از مدل EfficientNetB1 ویژگی‌های سطح بالا استخراج شد. پیش‌پردازش استاندارد مدل و تنظیم اندازه تصاویر قبل از ورود به شبکه انجام شد.

3. کلاس‌بندی

3-1. مدل یادگیری عمیق (EfficientNetB1)

مرحله 1: تنظیم مدل

- استفاده از مدل pretrained بر ImageNet
- بازگشایی یک‌سوم لایه‌های انتهایی و اعمال Fine-Tuning
- افزودن لایه‌های Dense و Dropout برای جلوگیری از بیش‌برازش

مرحله 2: تقسیم داده

تقسیم داده‌ها به:

- Train: 80%
- Validation: 10%
- Test: 10%

به صورت patient-wise برای جلوگیری از نشت داده.

مرحله ۳: رفع عدم تعادل کلاس‌ها با MI Augmentation

- محاسبه Mutual Information
- انتخاب تصاویر با MI پایین
- تولید نسخه‌های تقویت‌شده (Noise, Flip, Contrast)
- بالانس‌سازی مجموعه آموزش بدون استفاده از class_weight

مرحله 4: آموزش و ارزیابی

- آموزش با دستور fit
- ارزیابی با evaluate
- معیارهای دقت، AUC، F1-score گزارش شدند.

2-3. مدل‌های یادگیری ماشین

بر اساس ویژگی‌های رادیومیکس، مدل‌های زیر آموزش داده شدند:

- Random Forest
- LightGBM
- SVM
- MLP

پس از نرمال‌سازی ویژگی‌ها، مدل‌ها آموزش داده شده و معیارهای:

- Accuracy
- Precision
- Recall
- F1-score
- AUC

محاسبه و در یک جدول مقایسه گردید.

4. جدول جامع مقایسه مدل‌ها

در پایان نتایج مدل‌های:

- EfficientNetB1
- RandomForest
- LightGBM
- SVM
- MLP

در قالب یک جدول کلی تحلیل شد تا مشخص شود کدام روش در طبقه‌بندی سه‌کلاسه شدت چربی کبد عملکرد بهتری دارد.


```
## import cv2

import numpy as np

import matplotlib.pyplot as plt

# تعریف تابع بهبود محلی آماری
def statistical_local_enhancement(img, window_size=21, E=4.0, k0=1.2, k1=0.1,
k2=2.0):

    M = np.mean(img)
    global_std = np.std(img)
    img_f = img.astype(np.float32)

    # محاسبه میانگین محلی و انحراف معیار محلی در همسایگی
    mean_local = cv2.blur(img_f, (window_size, window_size))
    mean_sq_local = cv2.blur(img_f**2, (window_size, window_size))
    std_local = np.sqrt(np.maximum(0, mean_sq_local - mean_local**2))

    # اعمال شروط آماری برای تغییر پیکسل‌ها
    # شرط: میانگین محلی کوچکتر از درصدی از میانگین کل باشد و انحراف معیار محلی در بازه مشخصی باشد
    mask_mean = mean_local <= (k0 * M)
    mask_std = (std_local >= (k1 * global_std)) & (std_local <= (k2 * global_std))
    mask_final = mask_mean & mask_std

    # فقط در نواحی که شرایط را دارند E اعمال ضریب تقویت
```

```

enhanced_img = np.copy(img_f)
enhanced_img[mask_final] = enhanced_img[mask_final] * E

# بازگرداندن مقادیر به محدوده مجاز و فرمت uint8
return np.clip(enhanced_img, 0, 255).astype(np.uint8)

# قبلاً تعریف شده است X فرض بر این است که) انتخاب یک تصویر از دیتاست ۱.
idx = 0
img_raw = X[idx]

# تبدیل تصویر به فرمت استاندارد عددی ۲.
if isinstance(img_raw, np.ndarray) and img_raw.dtype == object:
    img_raw = img_raw.astype(np.float32)

img_float = np.array(img_raw, dtype=np.float32)
img = cv2.normalize(img_float, None, 0, 255, cv2.NORM_MINMAX)
img = np.uint8(img)

# اعمال روش‌های مختلف پردازش تصویر ۳.

# تصویر اول: اصلی
# تصویر دوم: Global Histogram Equalization
equalized = cv2.equalizeHist(img)

# تصویر سوم: Statistical Local Enhancement
# فراخوانی با پارامترهای منطقی تا تغییرات به وضوح دیده شوند
stat_local = statistical_local_enhancement(

```

```

img,
window_size=21,
E=2.0,
k0=1.2,
k1=0.1,
k2=2.0
)

# بهبود محلی پیشرفته برای مقایسه نهایی: CLAHE: تصویر چهارم
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_img = clahe.apply(img)

# لیست متغیرها برای رسم نمودار
images = [img, equalized, stat_local, clahe_img]
titles = ['Original Image', 'Global Hist Eq', 'Statistical Local', 'CLAHE']
colors = ['gray', 'blue', 'orange', 'green']

# تنظیم ابعاد کلی پنجره رسم
plt.figure(figsize=(20, 10))

# ردیف اول: نمایش تصاویر
for i in range(4):
    plt.subplot(2, 4, i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i], fontsize=12, fontweight='bold')
    plt.axis('off')

```

ایجاد ماسک از تصویر اصلی برای حذف پس‌زمینه در تمام تصاویر #

```
tissue_mask = img > 5
```

ردیف دوم: هیستوگرام‌ها #

```
for i in range(4):
```

```
    plt.subplot(2, 4, i+5)
```

```
    pixels_tissue = images[i][tissue_mask]
```

```
    plt.hist(pixels_tissue, bins=255, color=colors[i], alpha=0.7, log=True)
```

```
    plt.title(f"Histogram of {titles[i]}", fontsize=10)
```

```
    plt.xlabel('Pixel Intensity', fontsize=9)
```

```
    plt.ylabel('Frequency (Log Scale)', fontsize=9)
```

```
    plt.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

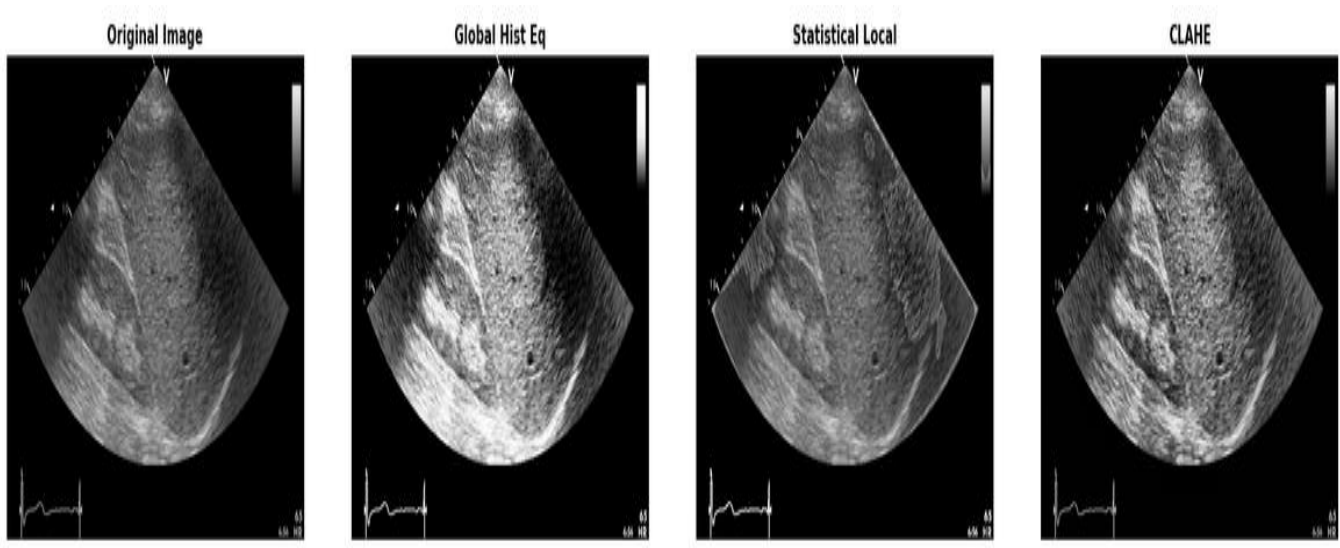
توضیحات کد :

- `import matplotlib.pyplot as plt ,import numpy as np ,import cv2`: فراخوانی کتابخانه‌های استاندارد پردازش تصویر (OpenCV)، محاسبات ماتریسی (NumPy) و رسم نمودار (Matplotlib).
- `idx = 0` و `img_raw = X[idx]`: انتخاب اولین تصویر (ایندکس 0) از مجموعه داده‌های تصاویر سونوگرافی (X).

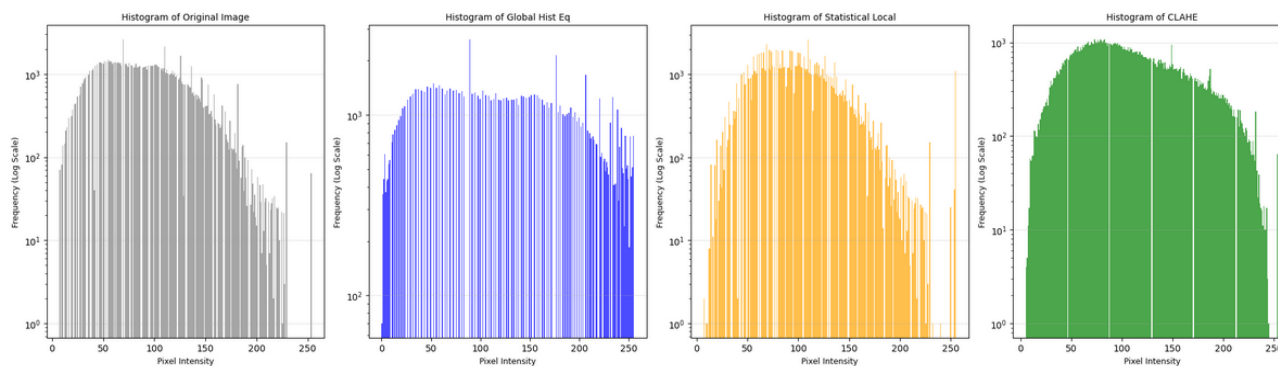
- `if isinstance(img_raw, np.ndarray) and img_raw.dtype == object`: یک شرط کنترلی برای زمانی که داده‌ها از فایل‌هایی مانند `mat` خوانده می‌شوند و ممکن است ساختار داده‌ای `object` داشته باشند.
- `img_raw = img_raw.astype(np.float32)` و `img_float = ...`: تبدیل اجباری داده‌های تصویر به فرمت استاندارد اعشاری برای جلوگیری از خطاهای محاسباتی در پردازش.
- `img = cv2.normalize(img_float, None, 0, 255, cv2.NORM_MINMAX)`: نرمال‌سازی مقادیر پیکسل‌های تصویر بین 0 تا 255 تا برای نمایش و پردازش با استانداردهای 8 بیتی سازگار شود.
- `img = np.uint8(img)`: تبدیل فرمت داده‌ها به عدد صحیح 8 بیتی بدون علامت که پیش‌نیاز توابع OpenCV است.
- `equalized = cv2.equalizeHist(img)`: اعمال روش (Global Histogram Equalization). این روش توزیع روشنایی را در کل تصویر یکنواخت می‌کند، اما معمولاً نویز را نیز در تصاویر پزشکی افزایش می‌دهد.
- `clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))`: ایجاد یک شیء از نوع CLAHE (تعدیل هیستوگرام تطبیقی با محدودیت کنتراست). پارامتر `tileGridSize` تصویر را به بلوک‌های 8×8 تقسیم می‌کند و `clipLimit=2.0` از افزایش بیش از حد نویز در نواحی همگن جلوگیری می‌کند.
- `clahe_img = clahe.apply(img)`: اعمال الگوریتم CLAHE روی تصویر اصلی.
- `images, titles, colors`: تعریف لیست‌هایی شامل تصاویر پردازش‌شده، عناوین آن‌ها و رنگ‌های نمودار برای استفاده در حلقه‌های رسم گراف.
- `plt.figure(figsize=(20, 10))`: ایجاد یک پنجره گرافیکی بزرگ با ابعاد 20 در 10 اینچ برای نمایش واضح تصاویر و نمودارها.
- `for i in range(4)` (اول): این حلقه تصاویر چهارگانه (اصلی، سراسری، آماری و CLAHE) را در ردیف بالایی تصویر (Subplot) نمایش می‌دهد. محورهای مختصات با `plt.axis('off')` حذف شده‌اند.

- `tissue_mask = img < 5`: ایجاد یک ماسک باینری. در تصاویر سونوگرافی، بخش بزرگی از تصویر را پس‌زمینه سیاه (مقدار نزدیک به صفر) تشکیل می‌دهد. این خط پیکسل‌هایی که مقدارشان بیشتر از ۵ است را به عنوان بافت واقعی بدن شناسایی می‌کند تا در رسم هیستوگرام، پیکسل‌های مزاحم پس‌زمینه لحاظ نشوند.
- حلقه `for i in range (4)` (دوم): این حلقه برای رسم هیستوگرام‌ها در ردیف پایین ایجاد شده است.
- `pixels_tissue = images[i][tissue_mask]`: اعمال ماسک بافت روی هر یک از تصاویر پردازش شده. با این کار، آرایه‌ای یک‌بعدی شامل فقط مقادیر شدت روشنایی بافت کبد استخراج می‌شود.
- `plt.hist(pixels_tissue, bins=255, ...)`: رسم هیستوگرام برای پیکسل‌های بافت با ۲۵۵ بازه (bin). تنظیمات شفافیت (`alpha`) و رنگ نیز اعمال می‌شود.
- توابع `plt.grid`, `plt.xlabel`, `plt.title`: تنظیمات ظاهری نمودارها شامل نام‌گذاری محورها و افزودن خطوط راهنما (Grid) برای خوانایی بهتر.
- `plt.tight_layout()` و `plt.show()`: تنظیم فاصله‌های بین نمودارها برای جلوگیری از تداخل متن‌ها و در نهایت نمایش پنجره گرافیکی.

نتیجه کد :



شکل 1



شکل 2

تحلیل و مقایسه روش‌های بهبود محلی:

مطابق صورت مسئله، دو روش اصلی بهبود تصویر مورد بررسی قرار گرفتند:

(۱) بهبود کنتراست سراسری با استفاده از Global Histogram Equalization و

(۲) بهبود کنتراست محلی مبتنی بر تحلیل آماری میانگین و واریانس در همسایگی‌های تصویر.

به منظور تحلیل بهتر نتایج و مقایسه کیفی روش‌های کلاسیک با روش‌های پیشرفته‌تر بهبود کنتراست در تصاویر پزشکی، روش CLAHE نیز به صورت تکمیلی در این بخش نمایش داده شده است.

همان‌طور که در نتایج شکل بالا مشخص است، روش بهبود آماری بر اساس میانگین و واریانس، توانسته بخش‌هایی از بافت را برجسته کند این روش اگرچه نسبت به روش سراسری بسیار کنترل‌شده‌تر عمل می‌کند، اما به دلیل استفاده از یک ضریب تقویت ثابت (E) در نواحی دارای شرط، همچنان در برخی از نقاط مستعد ایجاد اشباع‌شدگی خفیف است.

در مقابل، روش پیشرفته CLAHE دقیقاً نقطه ضعف روش آماری کلاسیک را پوشش داده است. این الگوریتم با اعمال محدودیت بر روی اوج‌های هیستوگرام (Clip Limit) و انجام درونیایی نرم بین بلوک‌های همسایه، از ایجاد نویزهای اشباع‌شده جلوگیری کرده است. در نتیجه، تصویر حاصل از روش CLAHE طبیعی‌ترین و

واضح‌ترین خروجی را برای تشخیص مرزهای کبد و پرده شکمی ارائه می‌دهد که آن را به بهترین گزینه برای مراحل بعدی استخراج ویژگی تبدیل می‌کند.

در این بخش، تاثیر سه رویکرد متفاوت برای بهبود کیفیت تصویر اولتراسوند (سونوگرافی) مقایسه شده است:

- **تصویر اصلی:** تصویر دارای کنتراست پایینی است و جزئیات بافت کبد و مرزهای آن به دلیل ماهیت تصویربرداری اولتراسوند، کمی کدر و درهم‌تنیده به نظر می‌رسند.
- **روش بهبود سراسری:** این روش کل تصویر را به طور یکسان روشن کرده است. همان‌طور که دیده می‌شود، با اینکه تصویر بسیار روشن‌تر شده، اما نویز ذاتی اولتراسوند نیز به شدت تقویت شده است. بافت‌ها حالت (Washed out) پیدا کرده‌اند و کاهش تمایز بین ساختارهای بافتی و افزایش نویز اسپکل مشاهده می‌شود.
- **روش بهبود محلی آماری:** با تنظیم ضریب تقویت $E=2$ ، این روش بر اساس میانگین و واریانس محلی عمل کرده است. برخلاف روش سراسری، این روش از روشن شدن بیش از حد کل تصویر جلوگیری کرده و فقط بافت‌هایی که شرایط آماری خاصی (تعریف شده در فرمول) داشتند را تغییر داده است. نتیجه این است که تصویر طبیعی‌تر مانده و از ایجاد لکه‌های نویزی شدید جلوگیری شده است (کنتراست ملایم).
- **روش پیشرفته (CLAHE):** بهترین نتیجه بصری را ارائه داده است. این روش با تقسیم تصویر به بلوک‌های کوچک و محدود کردن میزان تقویت کنتراست، توانسته مرز بافت‌ها را واضح کند، بدون اینکه نویز تصویر را مانند روش Global افزایش دهد.

تحلیل آماری و نمودارهای هیستوگرام

محور y به صورت لگاریتمی رسم شده است که باعث دید بهتر توزیع می‌شود.. تفسیر این نمودارها به این شکل است:

1. از آنجا که تصاویر سونوگرافی به شکل یک «مخروط» هستند و فضای اطراف این مخروط کاملاً سیاه (عدد صفر) است، تعداد پیکسل‌های سیاه بسیار زیاد است و باعث می‌شود بقیه نمودار فشرده به نظر برسد. بنابراین در تحلیل هیستوگرام از ماسک بافت استفاده شده است، پیکسل‌های پس‌زمینه سیاه حذف شده‌اند و هیستوگرام تنها توزیع شدت در بافت واقعی را نشان می‌دهد.

2. پخش‌شدگی پیکسل‌ها:

- در Original، تراکم پیکسل‌های بافت در نیمه سمت چپ (اعداد پایین‌تر و تاریک‌تر) است.
- در Global، خطوط آبی رنگ در سراسر محور افقی پخش شده‌اند. این یعنی فیلتر به زور تمام رنگ‌ها را در تصویر ایجاد کرده که دلیل اصلی نویزی شدن تصویر است.
- در Statistical Local، توزیع پیکسل‌ها (رنگ نارنجی) نسبت به حالت اصلی کمی بازتر شده اما مثل روش Global در کل محدوده کشیده نشده است. این نشان می‌دهد که الگوریتم کنترل‌شده عمل کرده است.
- در CLAHE، هیستوگرام (رنگ سبز) توزیع یکنواخت‌تر و نرم‌تری نسبت به روش آماری دارد که دلیل وضوح بالای تصویر چهارم است.

نتیجه‌گیری نهایی

مقایسه روش‌های مختلف نشان می‌دهد که روش Global Equalization برای تصاویر پزشکی مانند سونوگرافی مناسب نیست، زیرا نویزها را همراه با سیگنال اصلی به شدت تقویت می‌کند. در مقابل، روش محلی آماری پیاده‌سازی شده با استفاده از پارامترهای میانگین و واریانس (و تنظیم $E=2$) توانست بهبود کنتراست را به صورت انتخابی انجام دهد و از اشباع شدن تصویر جلوگیری کند. با این حال، الگوریتم CLAHE به دلیل استفاده از مکانیزم محدودکننده کنتراست و درون‌یابی بلوک‌ها، بهینه‌ترین خروجی را برای تشخیص مرزهای کبد و بافت‌های داخلی ارائه داد. مزیت روش آماری محلی این است که توانایی استخراج و نمایان کردن جزئیات پنهان در نواحی تاریک که ممکن است در سونوگرافی حاوی اطلاعات مهمی (مثل مرز یا بافت‌های عمیق‌تر) باشند و در مقابل مزیت CLAHE ایجاد یک تصویر متعادل‌تر، طبیعی‌تر و با نویز کمتر در کل تصویر.

```

import cv2

import numpy as np

import matplotlib.pyplot as plt

# آماده‌سازی تصویر ۱.
img = X[0]

if img.dtype != np.uint8:
    img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# برای شناسایی لبه‌های جهت‌دار - (Sobel) مشتق مرتبه اول ۲.
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
img_sobel = cv2.magnitude(sobel_x, sobel_y)
img_sobel = np.clip(img_sobel, 0, 255).astype(np.uint8)

# خواسته پروژه برای بهبود جزئیات - (Laplacian) مشتق مرتبه دوم ۳.
# لاپلاسیان حساسیت بالایی به تغییرات ناگهانی و نقاط ریز دارد
img_laplacian = cv2.Laplacian(img, cv2.CV_64F)
# (Sharpening) بهبود تصویر با استفاده از لاپلاسیان
# فرمول: Image_New = Image_Original - Laplacian
img_sharpened_lap = np.clip(img.astype(np.float32) - img_laplacian, 0,
255).astype(np.uint8)

# (ساده High-pass Filter) فیلتر فرکانسی ۴.
# در واقع یک فیلتر بالاگذر است (Unsharp Masking)

```

```

blur = cv2.GaussianBlur(img.astype(np.float32), (5,5), 1.0)
mask = img.astype(np.float32) - blur
img_high_pass = np.clip(img.astype(np.float32) + 1.5 * mask, 0, 255).astype(np.uint8)

# نمایش مقایسه‌ای نتایج ۵.
plt.figure(figsize=(20, 10))

titles = ["Original", "1st Derivative (Sobel)", "2nd Derivative (Laplacian Sharpen)", "High-
pass (Frequency)"]
images = [img, img_sobel, img_sharpened_lap, img_high_pass]

for i in range(4):
    plt.subplot(1, 4, i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

print("تحلیل مشتقات انجام شد")
print("- Sobel: لبه‌های ضخیم و اصلی کبد را مشخص کرد")
print("- Laplacian: جزئیات ریز و بافت‌های درونی کبد را تیزتر کرد (Sharper)")
print("- High-pass: کنتراست محلی تصویر را افزایش داد")

```

توضیحات کد :

cv2.Sobel : این تابع مشتق تصویر را می گیرد .

sobel_x : لبه های عمودی را شناسایی می کند (تغییرات در راستای افقی) .

sobel_y : لبه های افقی را شناسایی می کند (تغییرات در راستای عمودی) .

cv2.magnitude : برآیند دو بردار X و Y را می گیرد تا لبه ها در تمام جهات ترکیب شوند .

np.clip : اعداد را دوباره به بازه 0-255 برمی گرداند تا یک تصویر لبه دار استاندارد داشته باشیم.

cv2.Laplacian : مشتق مرتبه دوم را حساب می کند. لاپلاسیان به شدت به تغییرات ناگهانی (نقاط ریز و لبه های خیلی نازک) حساس است. در تصاویر پزشکی، برای پیدا کردن بافت های خیلی ریز استفاده می شود.

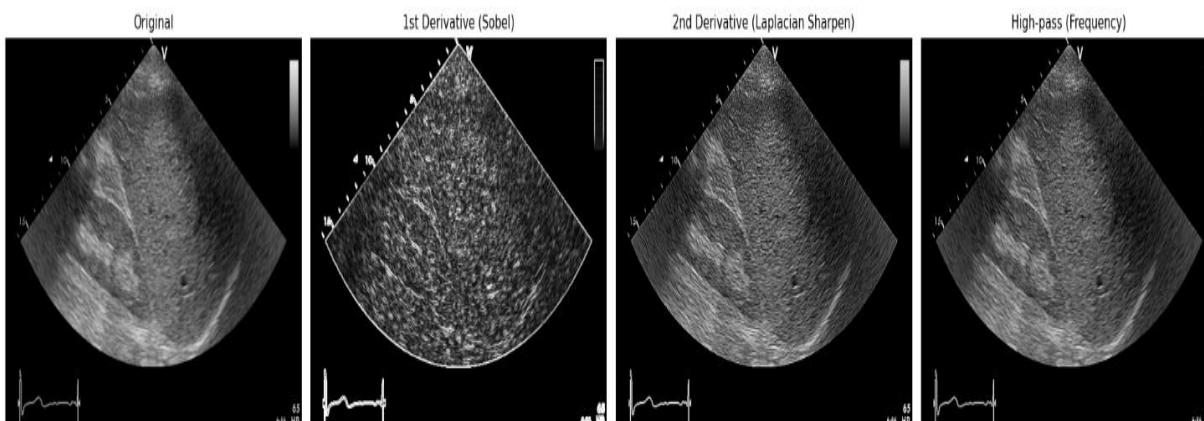
فرمول Sharpening : طبق اصول پردازش تصویر، اگر مشتق دوم (لاپلاسیان) را از تصویر اصلی کم کنیم، لبه ها تیزتر می شوند در واقع با این کار، اختلاف نوری لبه ها را دستکاری می کنیم تا چشم انسان آن ها را واضح تر ببیند.

GaussianBlur : یک تصویر تار ایجاد می کند که در واقع فرکانس های پایین (کلیات عکس) است .

mask : وقتی تصویر مات را از تصویر اصلی کم می کنیم، فقط فرکانس های بالا (جزئیات ریز و لبه ها) باقی می ماند.

در نهایت، آن جزئیات ریز (mask) را با یک ضریب (اینجا ۱.۵) دوباره به عکس اصلی اضافه می کنیم. نتیجه تصویری است که کنتراست محلی آن بالا رفته و بافت ها درشت تر دیده می شوند.

نتیجه کد :



✓ تحلیل مشتقات انجام شد:

- Sobel: لبه‌های ضخیم و اصلی کبد را مشخص کرد.
- Laplacian: جزئیات ریز و بافت‌های درونی کبد را تیزتر (Sharpen) کرد.
- High-pass: کنتراست محلی تصویر را افزایش داد.

شکل 3

در این بخش، هدف ارتقای وضوح و برجسته‌سازی جزئیات بافتی تصویر کبد با استفاده از فیلترهای مشتق‌گیر (در حوزه مکان) و فیلترهای بالاگذر (در حوزه فرکانس) بوده است. تحلیل نتایج به شرح زیر است:

۱. تحلیل مشتق مرتبه اول (فیلتر Sobel):

- **مشاهدات:** تصویر خروجی Sobel، مجموعه‌ای از خطوط و نقاط روشن در یک زمینه تیره است. این فیلتر شدت تغییرات ناگهانی (گرادیان) را محاسبه می‌کند. همان‌طور که در تصویر دیده می‌شود، لبه‌های ضخیم کبد و مرزهای اصلی بافت‌ها به خوبی شناسایی شده‌اند. اما چون در تصاویر سونوگرافی نویز Speckle زیاد است، این فیلتر تمام نویزها را به عنوان لبه شناسایی کرده و تصویر خروجی حالتی پر از نقطه پیدا کرده است.

۲. تحلیل مشتق مرتبه دوم (Laplacian Sharpening):

- **مشاهدات:** تصویر نسبت به حالت Original، بسیار «تیزتر» شده و دانه‌بندی‌های بافت کبد مشخص‌تر هستند.

اگرچه فیلتر لاپلاسین به دلیل مشتق مرتبه دوم بودن، نسبت به نویز حساسیت بیشتری دارد و نویز Speckle را در تصاویر سونوگرافی تقویت می‌کند، اما به دلیل توانایی بالا در تیز کردن (Sharpening) لبه‌های ریز، باعث می‌شود بافت‌های درونی کبد که در تصویر اصلی مات بودند، واضح‌تر به نظر برسند. این عمل باعث شده است که بافت کبد از حالت ابری و نرم خارج شده و ساختار دانه‌دانه آن برای بررسی‌های دقیق‌تر (مانند تشخیص کبد چرب) آماده شود.

۳. تحلیل فیلتر فرکانسی (High-pass Frequency Filter):

- **مشاهدات:** کنتراست محلی افزایش یافته و جزئیات بافتی با وضوح بسیار بالایی (حتی بیشتر از لاپلاسین) دیده می‌شوند.
- در این روش، فرکانس‌های پایین (که مربوط به بخش‌های یکنواخت و تاری تصویر هستند) حذف شده و فرکانس‌های بالا (که مربوط به لبه‌ها و بافت‌های ریز هستند) تقویت شده‌اند. خروجی نشان می‌دهد که این فیلتر به بهترین شکل توانسته است تغییرات جزئی در بافت کبد را آشکار کند. این فیلتر کنتراست بین نقاط روشن و تیره را در مقیاس کوچک افزایش داده که باعث می‌شود ویژگی‌های بافتی برای الگوریتم‌های هوش مصنوعی در مراحل بعدی، بسیار قابل تفکیک‌تر باشند.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# آماده‌سازی تصویر ۱.
img = X[0]
if img.dtype != np.uint8:
    img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# تعریف ماسک‌های مختلف لاپلاسیان ۲.
# ماسک استاندارد (بدون در نظر گرفتن قطرهای همسایگی)
kernel_1 = np.array([[0, 1, 0],
                      [1, -4, 1],
                      [0, 1, 0]])

# ماسک تقویت شده (با در نظر گرفتن همسایگی‌های قطری - لبه‌های دقیق‌تر)
kernel_2 = np.array([[1, 1, 1],
                      [1, -8, 1],
                      [1, 1, 1]])

# (مستقیم Sharpening برای) ماسک لاپلاسیان معکوس
kernel_3 = np.array([[-1, -1, -1],
                      [-1, 9, -1],
                      [-1, -1, -1]])

```

`[-1, -1, -1]))`

`# اعمال ماسک‌ها بر روی تصویر ۳.`

`lap_1 = cv2.filter2D(img, cv2.CV_64F, kernel_1)`

`lap_2 = cv2.filter2D(img, cv2.CV_64F, kernel_2)`

`img_sharpened = cv2.filter2D(img, cv2.CV_64F, kernel_3)`

`# تبدیل نتایج به فرمت قابل نمایش`

`lap_1_abs = np.clip(np.absolute(lap_1), 0, 255).astype(np.uint8)`

`lap_2_abs = np.clip(np.absolute(lap_2), 0, 255).astype(np.uint8)`

`img_sharpened = np.clip(img_sharpened, 0, 255).astype(np.uint8)`

`# نمایش و مقایسه نتایج ۴.`

`plt.figure(figsize=(20, 10))`

`plt.subplot(1, 4, 1)`

`plt.imshow(img, cmap='gray')`

`plt.title("Original Image")`

`plt.axis("off")`

`plt.subplot(1, 4, 2)`

`plt.imshow(lap_1_abs, cmap='gray')`

`plt.title("Basic Laplacian\n(4-Neighborhood)")`

`plt.axis("off")`

`plt.subplot(1, 4, 3)`

`plt.imshow(lap_2_abs, cmap='gray')`


```
plt.title("Extended Laplacian\n(8-Neighborhood)")
```

```
plt.axis("off")
```

```
plt.subplot(1, 4, 4)
```

```
plt.imshow(img_sharpened, cmap='gray')
```

```
plt.title("Sharpened Image\n(Using Laplacian Mask)")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

تحلیل برای گزارش ۵.

```
print("گزارش اعمال ماسک‌های لاپلاسین")
```

```
print("-ماسک 4-همسایگی: لبه‌های اصلی را با نویز کمتر استخراج کرد -")
```

```
print("-ماسک 8-همسایگی: جزئیات بیشتری از بافت داخل کبد را نمایان کرد اما نویز را هم افزایش داد -")
```

```
print("-با تقویت تفاوت پیکسل‌های مجاور، وضوح پرده شکمی را به طور محسوسی (Sharpen) ماسک ترکیبی -  
بهبود بخشید")
```

توضیحات کد :

```
kernel_1 = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]])
```

تعریف ماسک (کرنل) استاندارد لاپلاسین که تنها همسایگی‌های چهار جهت (بالا، پایین، چپ و راست) را در نظر می‌گیرد و برای تشخیص لبه‌های اصلی کاربرد دارد.

```
kernel_2 = np.array([[1, 1, 1], [1, -8, 1], [1, 1, 1]])
```

تعریف ماسک لاپلاسین توسعه یافته که همسایگی‌های 8 جهت (چهار طرف و چهار قطر) را لحاظ می‌کند و بنابراین لبه‌های ریزتر و جزئیات بیشتری را تشخیص می‌دهد، اما نویز بیشتری نیز ممکن است تولید کند.

```
kernel_3 = np.array([[ -1, -1, -1], [-1, 9, -1], [-1, -1, -1]])
```

تعریف ماسک لاپلاسین معکوس که برای افزایش وضوح تصویر به کار می‌رود. این ماسک تفاوت پیکسل با همسایگانش را تقویت می‌کند و باعث افزایش کنتراست لبه‌ها در تصویر می‌شود.

```
lap_1 = cv2.filter2D(img, cv2.CV_64F, kernel_1)
```

فیلتر کردن تصویر ورودی با هر سه ماسک لاپلاسین با استفاده از تابع `filter2D`

خروجی با نوع داده 64 بیت اعشاری (`float64`) است تا دقت محاسبات حفظ شود و نتایج منفی یا بزرگ را به طور صحیح نگه دارد.

```
lap_1_abs = np.clip(np.absolute(lap_1), 0, 255).astype(np.uint8)
```

به دلیل اینکه فیلتر لاپلاسین مقادیر منفی تولید می‌کند، ابتدا قدر مطلق نتایج گرفته می‌شود تا فقط مقدار شدت لبه‌ها باقی بماند.

سپس با استفاده از `clip` مقادیر در بازه قانونی 0 تا 255 محدود و در نهایت به نوع داده 8 بیتی (`uint8`) تبدیل می‌شوند تا برای نمایش مناسب باشند.

```
plt.figure(figsize=(20, 10))
```

.....

```
plt.show()
```

ایجاد یک پنجره نمایش بزرگ با اندازه ۱۰×۲۰ اینچ.

چهار تصویر کنار هم به صورت مقایسه‌ای در یک ردیف قرار می‌گیرند:

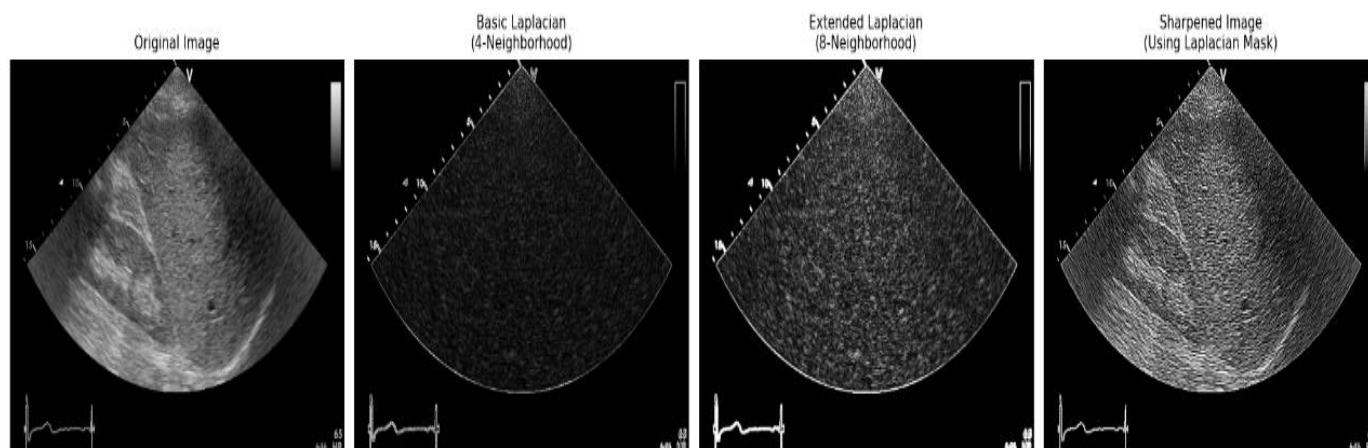
1. تصویر اصلی،

2. تصویر پس از اعمال ماسک پایه لاپلاسین (4 همسایگی)،

3. تصویر پس از اعمال ماسک توسعه یافته لاپلاسین (8 همسایگی)،

4. تصویر شارپ شده توسط ماسک مخصوص لاپلاسین.

نتیجه کد :



شکل 4

✓ گزارش اعمال ماسک‌های لاپلاسین

- ماسک 4-همسایگی: لبه‌های اصلی را با نویز کمتر استخراج کرد.
- ماسک 8-همسایگی: جزئیات بیشتری از بافت داخل کبد را نمایان کرد اما نویز را هم افزایش داد.
- با تقویت تفاوت پیکسل‌های مجاور، وضوح پرده شکمی را به طور محسوسی بهبود بخشید: (Sharpen) ماسک ترکیبی -

در این بخش از پروژه، عملکرد سه نوع ماسک مشتق‌گیر مرتبه دوم (لاپلاسین) بر روی تصاویر سونوگرافی کبد ارزیابی شد. هدف، بررسی تاثیر همسایگی‌های مختلف در استخراج جزئیات و همچنین بهبود وضوح (Sharpening) تصویر بوده است. نتایج به شرح زیر است:

۱. ماسک لاپلاسین پایه (4-Neighborhood)

- کرنل استفاده شده: کرنلی که در آن فقط 4 همسایه افقی و عمودی در نظر گرفته می‌شوند (مرکز -4).
- تحلیل تصویر خروجی: همان‌طور که در تصویر Basic Laplacian مشاهده می‌شود، این فیلتر لبه‌های اصلی و تغییرات شدید شدت روشنایی را استخراج کرده است. به دلیل محدودیت در محاسبه فقط 4

جهت، میزان نویز Speckle استخراج شده نسبتاً کمتر است، اما در مقابل، بسیاری از جزئیات ظریف بافتی نیز نادیده گرفته شده‌اند. این حالت تصویر تاریک‌تری ایجاد کرده است.

۲. ماسک لاپلاسین توسعه‌یافته (Neighborhood-8)

- کرنل استفاده شده: کرنلی که در آن هر 8 همسایه (شامل همسایه‌های قطری) در نظر گرفته می‌شوند (مرکز -8).
- تحلیل تصویر خروجی: در تصویر Extended Laplacian، مشهود است که با احتساب همسایه‌های قطری، فیلتر به تغییرات در تمام جهات حساس شده است. این امر باعث شده تا جزئیات بسیار ریزتری از بافت پارانشیم کبد نمایان شود و ساختار دانه‌بندی بافت بهتر مشخص گردد. با این حال، همان‌طور که انتظار می‌رود، افزایش حساسیت به تمام جهات منجر به افزایش چشمگیر نویز (به صورت نقاط سفید متراکم) در خروجی شده است.

۳. ماسک لاپلاسین برای وضوح‌افزایی (Sharpening Mask)

- کرنل استفاده شده: کرنل ترکیبی (مرکز 9) که در واقع حاصل تفریق لاپلاسین ۸-همسایگی از تصویر اصلی است (معادل $(g(x,y)=f(x,y)-\nabla^2 f(x,y))$)
- تحلیل تصویر خروجی: در تصویر Sharpened Image، به جای استخراج صرف لبه‌ها، جزئیات استخراج‌شده توسط مشتق دوم به تصویر اصلی اضافه شده‌اند. نتیجه این عملیات، بهبود قابل توجه وضوح تصویر (Sharpening) است. خطوط مرزی، ساختارهای داخلی و به ویژه پرده‌ها و بافت‌های متراکم‌تر با وضوح و کنتراست بالاتری نسبت به تصویر اصلی دیده می‌شوند. این تصویر برای بررسی‌های بالینی و تشخیص بافت‌های غیرطبیعی ارزش بیشتری دارد.

نتیجه‌گیری

استفاده از ماسک‌های لاپلاسین نشان داد که افزایش تعداد همسایگی‌ها در محاسبه مشتق دوم، باعث افزایش استخراج جزئیات بافتی به قیمت افزایش نویز می‌شود. همچنین ترکیب تصویر اصلی با خروجی لاپلاسین (Sharpening)، روشی بسیار موثر برای ارتقای وضوح بصری تصاویر سونوگرافی، علیرغم ماهیت نویزی آن‌ها، محسوب می‌شود.

```

import numpy as np
import cv2
import matplotlib.pyplot as plt

# آماده‌سازی تصویر اصلی ۱.
img = X[0] # شامل تصاویر خاکستری است X فرض بر اینکه
if img.dtype != np.uint8:
    img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

rows, cols = img.shape

# ۲. (Gaussian) ایجاد نویز گوسی
# مدل‌سازی نویز سنسور در شرایط کم‌نور
mean = 0
sigma = 25
gauss = np.random.normal(mean, sigma, (rows, cols))
gaussian_img = np.clip(img + gauss, 0, 255).astype(np.uint8)

# ۳. (Salt & Pepper) ایجاد نویز فلفل‌نمکی
# مدل‌سازی خطاهای دیجیتال و پیکسل‌های سوخته
sp_img = img.copy()
prob = 0.05 # درصد تصاویر نویز داشته باشد 5
num_salt = np.ceil(prob * img.size * 0.5)
num_pepper = np.ceil(prob * img.size * 0.5)

```

```

# افزودن نمک (سفید)
coords = [np.random.randint(0, i - 1, int(num_salt)) for i in img.shape]
sp_img[tuple(coords)] = 255

# افزودن فلفل (سیاه)
coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in img.shape]
sp_img[tuple(coords)] = 0

# ۴. (Exponential) ایجاد نویز نمایی
# مدل‌سازی پدیده‌های بازتابی خاص یا تغییرات ناگهانی شدت در نواحی همگن
exp_noise = np.random.exponential(scale=20.0, size=(rows, cols))
exp_img = np.clip(img + exp_noise, 0, 255).astype(np.uint8)

# نمایش ترکیبی تصاویر و هیستوگرام‌ها. ۵
fig, axes = plt.subplots(2, 4, figsize=(22, 10))

titles = ["Original", "Gaussian", "Salt & Pepper", "Exponential"]
images = [img, gaussian_img, sp_img, exp_img]
colors = ['black', 'blue', 'red', 'green']

for i in range(4):
    # ردیف اول: تصاویر
    axes[0, i].imshow(images[i], cmap='gray')
    axes[0, i].set_title(f"Image: {titles[i]}", fontsize=14)
    axes[0, i].axis('off')

    # ردیف دوم: هیستوگرام‌ها

```

```
axes[1, i].hist(images[i].ravel(), bins=256, color=colors[i], alpha=0.7, range=(0, 256))
axes[1, i].set_title(f"Histogram: {titles[i]}", fontsize=12)
axes[1, i].set_yscale('log') # برای دیده شدن بهتر نقاط نویز
```

```
plt.tight_layout()
plt.show()
```

```
print("تحلیل نویزها:")
print("- نویز گوسی: باعث پهن تر شدن و دنداندار شدن هیستوگرام حول میانگین تصویر می شود -")
print("- نویز فلفل نمکی: دو ستون بلند در ۰ و ۲۵۵ ایجاد می کند (نویز ضربه ای با نقاط سفید و سیاه) -")
print("- نویز نمایی: هیستوگرام را به سمت راست (شدت های روشن تر) متمایل کرده و درخشش موضعی تصویر را افزایش می دهد")
```

توضیحات کد :

```
if img.dtype != np.uint8: img = cv2.normalize(img, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
```

بررسی می شود که نوع داده تصویر uint8 باشد .

اگر نباشد :

- با استفاده از cv2.normalize شدت پیکسل ها به بازه ۰ تا ۲۵۵ تبدیل می شود.

- سپس به نوع uint8 تبدیل می شود.

```
rows, cols = img.shape
```

تعداد سطر ها و ستون های تصویر استخراج می شود .

این ابعاد برای تولید نویز با همان اندازه تصویر استفاده می شوند.

```
gauss = np.random.normal(mean, sigma, (rows, cols))
```

تولید نویز تصادفی با توزیع نرمال

prob = 0.05 : 5% از پیکسل‌ها نویزدار می‌شوند.

```
num_salt = np.ceil(prob * img.size * 0.5)
```

```
num_pepper = np.ceil(prob * img.size * 0.5)
```

تعداد پیکسل‌های سفید (Salt)

تعداد پیکسل‌های سیاه (Pepper)

```
coords = [np.random.randint(0, i - 1, int(num_salt)) for i in img.shape]
```

افزودن نمک (پیکسل سفید) و مختصات تصادفی تولید می‌شود.

```
coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in img.shape]
```

افزودن فلفل (پیکسل سیاه) مختصات تصادفی جدید تولید می‌شود.

```
exp_noise = np.random.exponential(scale=20.0, size=(rows, cols))
```

تولید نویز با توزیع نمایی و پارامتر scale برابر با 1/λ است.

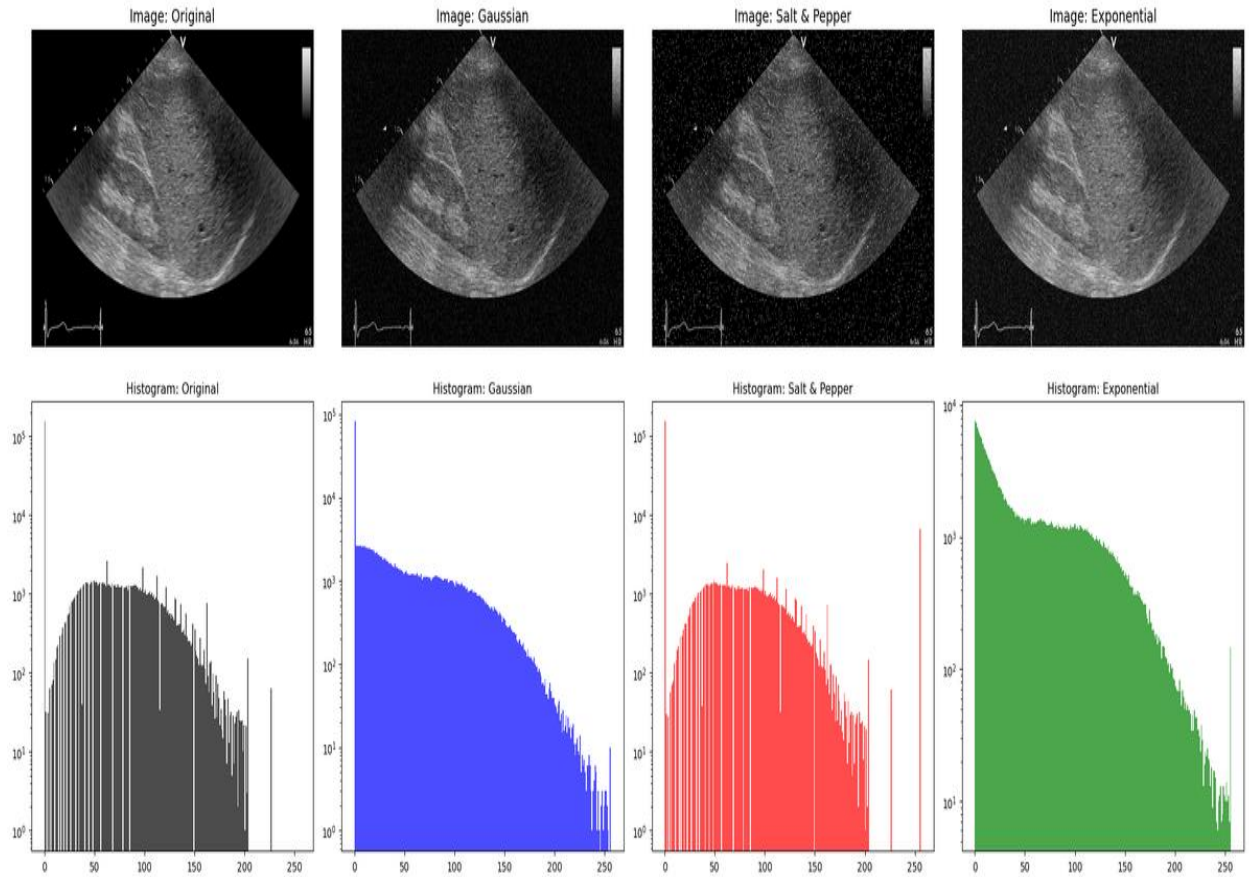
این توزیع فقط مقادیر مثبت تولید می‌کند.

```
exp_img = np.clip(img + exp_noise, 0, 255).astype(np.uint8)
```

نویز نمایی به تصویر افزوده می‌شود.

چون نویز فقط مثبت است، تصویر به سمت روشن‌تر شدن متمایل می‌شود.

در این بخش، تاثیر نویزهای مختلف بر بافت کبد و هیستوگرام آن تحلیل می‌شود:



✓ تحلیل نویزها:

- نویز گوسی: باعث پهن‌تر شدن و دنداندار شدن هیستوگرام حول میانگین تصویر می‌شود.
- نویز فل‌فل‌نمکی: دو ستون بلند در ۰ و ۲۵۵ ایجاد می‌کند (نویز ضربه‌ای با نقاط سفید و سیاه).
- نویز نمایی: هیستوگرام را به سمت راست (شدت‌های روشن‌تر) متمایل کرده و درخشش موضعی تصویر را افزایش می‌دهد.

شکل 5

الف) نویز گوسی (Gaussian Noise)

- تحلیل تصویر: تصویر دچار یک حالت برفک یکنواخت شده است. جزئیات ظریف بافت پارانشیم کبد در زیر این نویز پنهان شده‌اند.
- تحلیل هیستوگرام: هیستوگرام نسبت به حالت اصلی پهن‌تر شده و لبه‌های دنداندار پیدا کرده است. این نشان‌دهنده اضافه شدن مقادیر تصادفی با توزیع نرمال به تمام سطوح خاکستری است.

ب) نویز فل‌فل‌نمکی (Pepper & Salt)

- تحلیل تصویر: پیکسل‌های سیاه و سفید به صورت نقطه‌ای روی تصویر ظاهر شده‌اند.

- **تحلیل هیستوگرام:** بارزترین ویژگی، ایجاد دو ستون بسیار بلند در ابتدا (۰) و انتها (۲۵۵) هیستوگرام است. به دلیل استفاده از مقیاس لگاریتمی، این دو قله به وضوح در نمودار قرمز دیده می‌شوند.

(ج) نویز نمایی (Exponential)

- **تحلیل تصویر:** این نویز که ماهیت افزایشی دارند، مقادیر پیکسل‌ها را بالا برده و باعث کاهش شدید کنتراست می‌شوند. تصویر به سمت روشنایی کاذب و می‌رود. در سونوگرافی، این نویزها (Speckle) را شبیه‌سازی می‌کنند.
- **تحلیل هیستوگرام:** با اضافه شدن نویز، پیکسل‌های مشکی پس‌زمینه از صفر مطلق خارج شده و مقادیر بیشتری (روشن‌تری) می‌گیرند. این امر باعث ایجاد یک قله‌ی عظیم در مقادیر پایین روشنایی شده است. در نتیجه، فرم کلی هیستوگرام به یک توزیع (Right-skewed) تبدیل شده و توزیع پیکسل‌ها در فرکانس‌های بالاتر افزایش یافته که نشان‌دهنده غلبه نویز بر سیگنال اصلی است.

نتیجه‌گیری نهایی:

بررسی هیستوگرام‌ها ثابت می‌کند که هر نوع نویز در تصویر، امضای خاصی در توزیع آماری پیکسل‌ها بر جای می‌گذارد. در تصاویر پزشکی سونوگرافی، نویزهای پارامتری (مثل نمایی) مخرب‌ترین اثر را روی مرز بافت‌ها دارند و حذف آن‌ها نیازمند فیلترهای تطبیقی است.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# تصویر اصلی
img = X[0]
if img.dtype != np.uint8:
    img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# 1) Median Filter: حذف نویزهای ضربه‌ای (S&P) با حفظ لبه‌ها
median = cv2.medianBlur(img, 11)

# 2) Gaussian Filter: حذف نویزهای گوسی و نرم‌سازی کلی بافت
gaussian = cv2.GaussianBlur(img, (11,11), 0)

# 3) Laplacian Sharpening: (جزئیات ریز) تقویت لبه‌های مرتبه دوم
laplacian = cv2.Laplacian(img, cv2.CV_64F)
# فرمول استاندارد تقویت لبه
lap_sharp = np.clip(img.astype(np.float32) - 1 * laplacian, 0, 255).astype(np.uint8)

# 4) Unsharp Masking: افزایش کنتراست لبه‌ها با استفاده از ماسک تار
blur_for_unsharp = cv2.GaussianBlur(img, (15,15), 3.0)
unsharp = cv2.addWeighted(img, 2.0, blur_for_unsharp, -1.0, 0)

```

نمایش نتایج

```
plt.figure(figsize=(15, 10))
```

```
images = [img, median, gaussian, lap_sharp, unsharp]
```

```
titles = ["Original", "Median (Denoise)", "Gaussian (Smooth)", "Laplacian (Detail)",  
"Unsharp (Contrast)"]
```

```
for i in range(5):
```

```
    plt.subplot(2, 3, i+1)
```

```
    plt.imshow(images[i], cmap='gray')
```

```
    plt.title(titles[i])
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

توضیحات کد :

`median = cv2.medianBlur(img, 11)` : فیلتر میانه با پنجره 11 در 11. این فیلتر غیرخطی است و مقدار هر پیکسل را با میانه مقادیر همسایگی اش جایگزین می کند .

`gaussian = cv2.GaussianBlur(img, (11,11), 0)` : فیلتر پایین گذر گوسی با پنجره 11 در 11. پیکسل ها را بر اساس یک توزیع زنگوله ای (نرمال) میانگین گیری می کند که باعث محو شدن یکنواخت تصویر می شود .

`laplacian = cv2.Laplacian(img, cv2.CV_64F)` : محاسبه مشتق دوم مکان (لاپلاسین). از فرمت 64 بیتی اعشاری استفاده شده تا مقادیر منفی (که نشان دهنده شیب لبه ها هستند) از بین نروند .

`lap_sharp = np.clip(img.astype(np.float32) - 1 * laplacian, 0, (255.astype(np.uint8))` : این خط عمل Sharpening (تیز کردن) را انجام می دهد. تصویر اصلی منهای کسری از ماسک لاپلاسین می شود تا لبه ها تقویت شوند. تابع `clip` تضمین می کند که مقادیر پیکسل ها از بازه 0 تا 255 خارج نشوند .

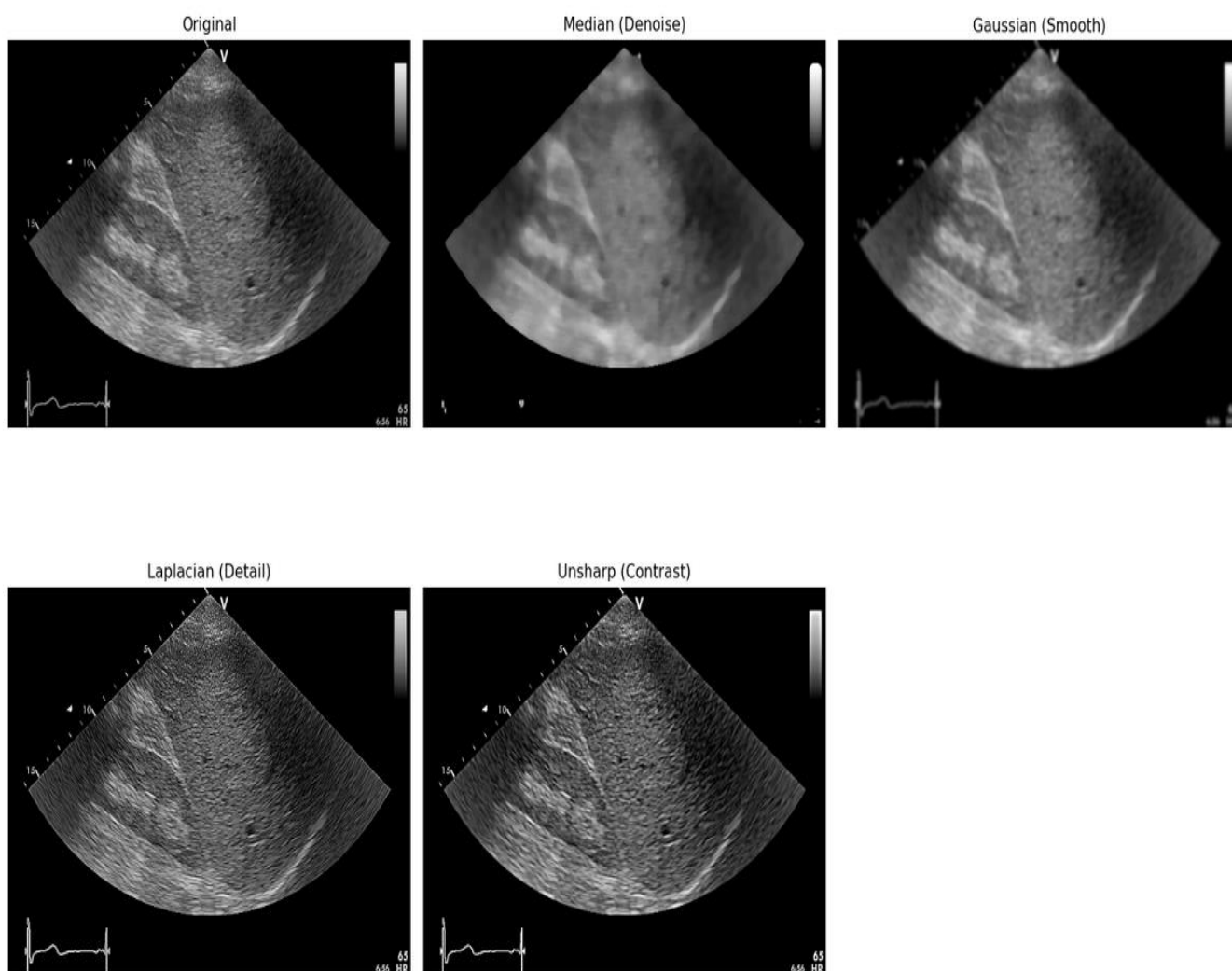
، $\text{blur_for_unsharp} = \text{cv2.GaussianBlur}(\text{img}, (15,15), 3.0)$: برای فیلتر Unsharp Masking

ابتدا یک نسخه تار از تصویر اصلی می‌سازیم .

فرمول ماسک آن‌شارپ به $\text{unsharp} = \text{cv2.addWeighted}(\text{img}, 2.0, \text{blur_for_unsharp}, -1.0, 0)$

صورت $\text{Original} + (\text{Original} - \text{Blur})$ است. تابع addWeighted دقیقاً همین کار را می‌کند $2 \times \text{img} - 1 \times \text{blur2}$. این کار لبه‌های فرکانس بالا را به تصویر برمی‌گرداند .

نتیجه‌کد:



شکل 6

بررسی و مقایسه 4 فیلتر اثرگذار در بهبود تصاویر سونوگرافی

در تصاویر سونوگرافی کبد، دو چالش اصلی وجود دارد: (۱) وجود نویز ذاتی Speckle که بافت را دانه‌دار می‌کند و (۲) نیاز به وضوح بالا در مرز بین بافت‌ها برای تشخیص پزشکی. برای بهبود تصویر، ۴ فیلتر با رویکردهای متفاوت (نرم‌سازی و تیزسازی) اعمال و با تصویر اصلی مقایسه شدند:

1. فیلتر میانه (Median Filter - Denoise):

- **استدلال:** این فیلتر غیرخطی، مقادیر پرت را در یک همسایگی محلی حذف می‌کند. در تصاویر پزشکی، این فیلتر برای حذف نویزهای ضربه‌ای بسیار کارآمد است زیرا بر خلاف فیلترهای میانگین‌گیر، مرزها و لبه‌های بافت را به شدت تار نمی‌کند.
- **مقایسه با تصویر اصلی:** در تصویر خروجی Median، مشاهده می‌شود که دانه‌بندی‌های ریز و نویزهای نقطه‌ای کاهش یافته و بافت کبد یکدست‌تر شده است، در حالی که خطوط مرزی همچنان وضوح قابل قبولی دارند.

2. فیلتر گوسی (Gaussian Filter - Smooth):

- **استدلال:** یک فیلتر پایین‌گذر خطی است که فرکانس‌های بالا (تغییرات ناگهانی پیکسل‌ها) را فیلتر می‌کند. این فیلتر نویزهای با توزیع نرمال را کاهش داده و یک هموارسازی سراسری ایجاد می‌کند.
- **مقایسه با تصویر اصلی:** در نتیجه‌ی Gaussian، کل تصویر نرم‌تر و محوتر شده است. جزئیات ظریف بافت پارانشیم کبد کاهش یافته‌اند. این فیلتر به تنهایی برای تشخیص بالینی مناسب نیست، اما به عنوان پیش‌پردازش (برای کاهش نویز قبل از استخراج لبه) بسیار مفید است.

3. فیلتر لاپلاسی (Laplacian Sharpening - Detail):

- **استدلال:** لاپلاسین بر مبنای مشتق دوم مکانی عمل می‌کند و به تغییرات شدید شدت روشنایی (لبه‌ها) بسیار حساس است. با تفریق مشتق دوم از تصویر اصلی، لبه‌ها و جزئیات ریز به شدت تقویت می‌شوند (High-pass filtering).

- **مقایسه با تصویر اصلی:** خروجی Laplacian بسیار واضح تر و تیزتر از تصویر اصلی است. مرزهای داخلی و خطوط آناتومیک برجسته تر شده اند. اما از آنجا که مشتق دوم به نویز بسیار حساس است، می توان دید که نویزهای پس زمینه (Speckle) نیز همراه با لبه ها تقویت شده و تصویر کمی خشن تر به نظر می رسد.

4. (Unsharp Masking - Contrast):

- **استدلال:** این روش ترکیبی از هموارسازی و تیز کردن است. در این روش، لبه ها با کم کردن یک نسخه تار شده از تصویر اصلی استخراج شده و سپس برای افزایش کنتراست محلی، به تصویر اصلی اضافه می شوند.
- **مقایسه با تصویر اصلی:** تصویر Unsharp بهترین تعادل را بین افزایش وضوح و حفظ بافت ارائه داده است. کنتراست کلی تصویر (به ویژه در نواحی تاریک تر و مرزهای بافت) نسبت به تصویر اصلی بهبود چشمگیری یافته است، بدون اینکه مانند روش لاپلاسین نویزهای ریز به صورت افراطی تقویت شوند.

نتیجه گیری:

برای اهداف تشخیصی در سونوگرافی، فیلتر Unsharp Masking برای برجسته کردن مرزها بدون تخریب شدید بافت، و فیلتر Median برای کاهش نویز بدون تار کردن لبه ها، اثربخش ترین نتایج را نسبت به تصویر اصلی از خود نشان دادند.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

sample_indices = [0, 50, 100]
plt.figure(figsize=(18, 12))

for k, idx in enumerate(sample_indices):
    img = X[idx]

    if img.dtype != np.uint8:
        img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

    # حذف نویز 1.
    denoised = cv2.medianBlur(img, 11)

    # 2. Threshold
    _, binary_region = cv2.threshold(denoised, 100, 255, cv2.THRESH_BINARY)

    # 3. Morphological Closing
    kernel = np.ones((7,7), np.uint8)
    closed = cv2.morphologyEx(binary_region, cv2.MORPH_CLOSE, kernel)

    # 4. Connected Components

```



```

num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(closed)

cleaned = np.zeros_like(closed)

min_area = 80 # حداقل اندازه ناحیه معتبر

for i in range(1, num_labels): # label صفر پس زمینه است
    area = stats[i, cv2.CC_STAT_AREA]
    if area > min_area:
        cleaned[labels == i] = 255

# 5. استخراج مرز
final_borders = cv2.morphologyEx(cleaned, cv2.MORPH_GRADIENT, np.ones((3,3),
np.uint8))

# نمایش
plt.subplot(3, 3, 3*k+1)
plt.imshow(img, cmap='gray')
plt.title("Original")

plt.subplot(3, 3, 3*k+2)
plt.imshow(cleaned, cmap='gray')
plt.title("Internal Regions")

plt.subplot(3, 3, 3*k+3)
plt.imshow(final_borders, cmap='gray')
plt.title("Accurate Boundaries")

```

```
for i in range(1,4):  
    plt.subplot(3,3,3*k+i).axis('off')
```

```
plt.tight_layout()  
plt.show()
```

توضیحات کد :

: `img = cv2.normalize(...)`

نرمال سازی : مقادیر پیکسل تصویر را به بازه استاندارد [255, 0] می برد تا فرآیندهای بعدی نسبت به تغییرات جزئی روشنایی مقاوم باشند.

: `cv2.medianBlur(img, 11)`

فیلتر میانه : حذف نویز Speckle سونوگرافی. این فیلتر به دلیل حفظ لبه های تیز ارگان ها، بسیار بهتر از فیلترهای میانگین گیر عمل می کند.

: `cv2.threshold(..., 100, ...)`

آستانه گذاری : تبدیل تصویر خاکستری به یک ماسک باینری. با تعیین مقدار 100، بافت های پرچگالی (بخش های روشن تر) به عنوان کاندیدای نواحی هدف جداسازی می شوند.

: `cv2.morphologyEx(..., CLOSE, ...)`

عملیات بستن (Closing) : ترکیبی از (Dilation) و (Erosion) این مرحله سوراخ های ریز داخل نواحی سفید را پر کرده و مرزهای شکسته را به هم متصل می کند تا ساختاری یکپارچه ایجاد شود.

: `cv2.connectedComponentsWithStats(...)`

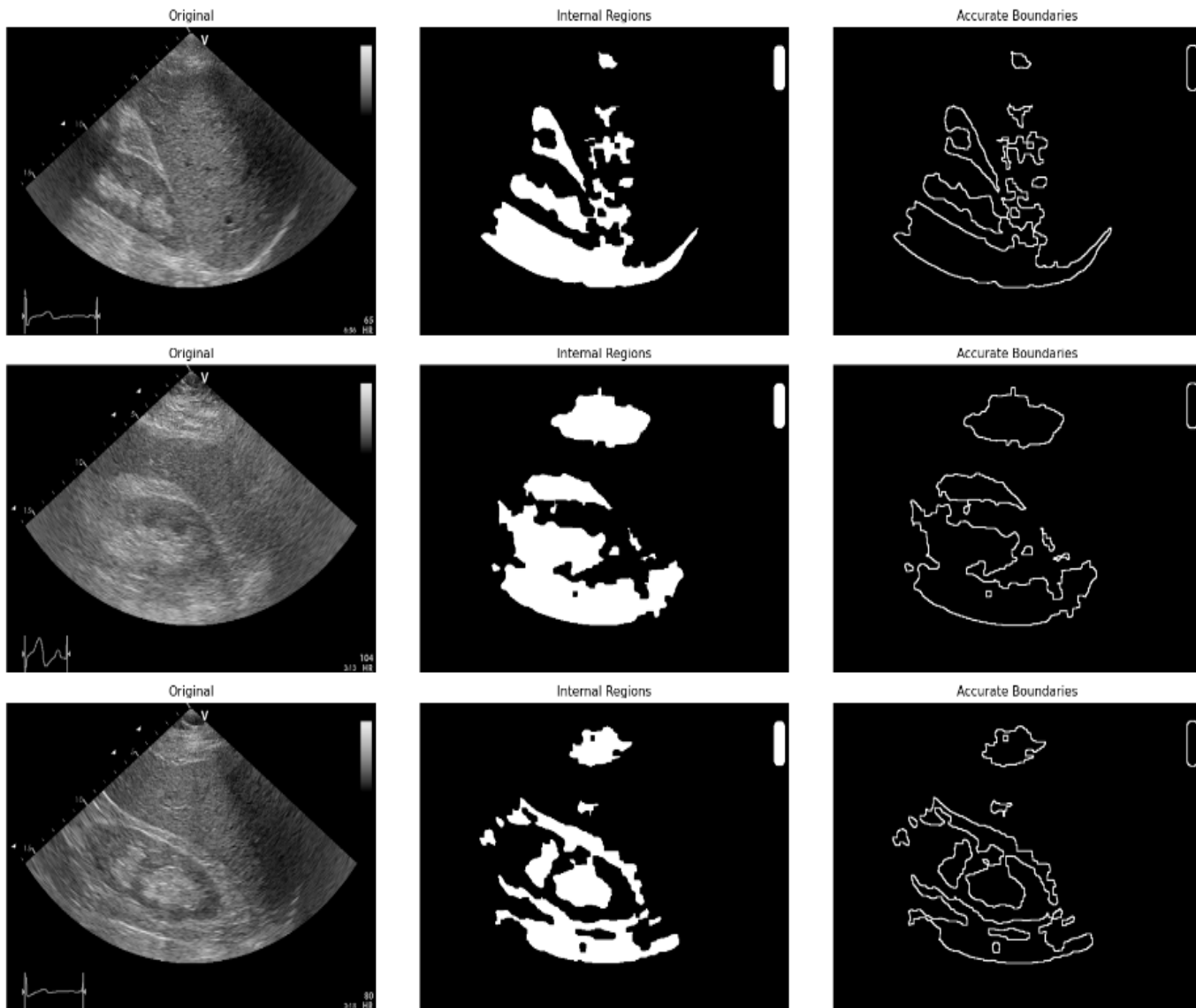
تحلیل اجزای متصل : شناسایی تمام جزایر سفید در ماسک. این ابزار به ما اجازه می دهد نواحی کوچک و نویزهای باقیمانده را بر اساس مساحت (Area) شناسایی و حذف کنیم.

: if area > min_area: ..

فیلتر مساحت : حذف نویزهای ناخواسته که بسیار کوچکتر از اندازه آناتومیک ارگان هستند تا فقط نواحی اصلی و معنادار باقی بمانند.

: cv2.morphologyEx(..., GRADIENT, ...)

گرادیان مورفولوژیک : استخراج دقیق لبه‌های خارجی و داخلی نواحی به دست آمده از طریق محاسبه تفاضل بین تصویر متسع شده و فرسایش یافته.



شکل 7

تفسیر نتایج :

خروجی حاصل دارای سه ویژگی کلیدی است :

پیوستگی مرزها : برخلاف روش‌های لبه‌یابی ساده مثل Canny که لبه‌ها را تکه‌تکه نشان می‌دهند، مرزهای استخراج شده در اینجا کاملاً بسته و محیطی هستند. این یعنی الگوریتم شکل ارگان را درک کرده است.

تفکیک آناتومیک : همانطور که در ستون دوم (Internal Regions) مشاهده می‌شود، الگوریتم موفق شده است توده‌های اصلی بافت را از نويز محیطی تفکیک کند. این نشان می‌دهد که پارامترهای انتخاب شده (مثل مساحت 80 پیکسل) به درستی مرز بین نويز و بافت واقعی را تشخیص داده‌اند.

بهینه‌سازی نويز : با مقایسه تصویر Original و Accurate Boundaries مشاهده می‌شود که نويزهای پس‌زمینه حذف شده و فقط ساختارهای آناتومیک مرزبندی شده‌اند.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# تصویر نمونه
img = X[0]
if img.dtype != np.uint8:
    img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# کاهش نویز با فیلتر میانه ۱.

blur = cv2.medianBlur(img, 7)
# اعمال فیلتر لاپلاسیان (استخراج لبه‌های مرتبه دوم). ۲.
laplacian = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
laplacian_abs = np.uint8(np.absolute(laplacian))

# آستانه‌گذاری خودکار برای حذف نویزهای پس‌زمینه و تثبیت لبه‌ها ۳.
# برای پیدا کردن بهترین حد آستانه به صورت خودکار Otsu استفاده از
_, edges_otsu = cv2.threshold(laplacian_abs, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

# نمایش نتایج ۴.
plt.figure(figsize=(16, 5))

titles = ["Original", "Gaussian Blur", "Laplacian Raw", "Final Edges (Otsu)"]

```

```
images = [img, blur, laplacian_abs, edges_otsu]
```

```
for i in range(4):
```

```
    plt.subplot(1, 4, i+1)
```

```
    plt.imshow(images[i], cmap='gray')
```

```
    plt.title(titles[i])
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("لبه‌یابی لاپلاسیان با موفقیت انجام شد")
```

توضیحات کد :

```
: blur = cv2.medianBlur(img, 7)
```

فیلتر لاپلاسیان به شدت به نویز حساس است (چون مشتق مرتبه دوم نوسانات کوچک را بزرگ‌نمایی می‌کند) .

استفاده از فیلتر میانه با پنجره 7×7 باعث می‌شود نویزهای دانه‌دانه سونوگرافی حذف شوند در حالی که (Sharpness) لبه‌های اصلی حفظ شود .

```
: laplacian = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
```

اپراتور لاپلاسیان مشتق دوم تصویر را محاسبه می‌کند .

از CV_64F استفاده شده تا مقادیر منفی (که در لبه‌ها رخ می‌دهد) حفظ شوند و از دست نروند .

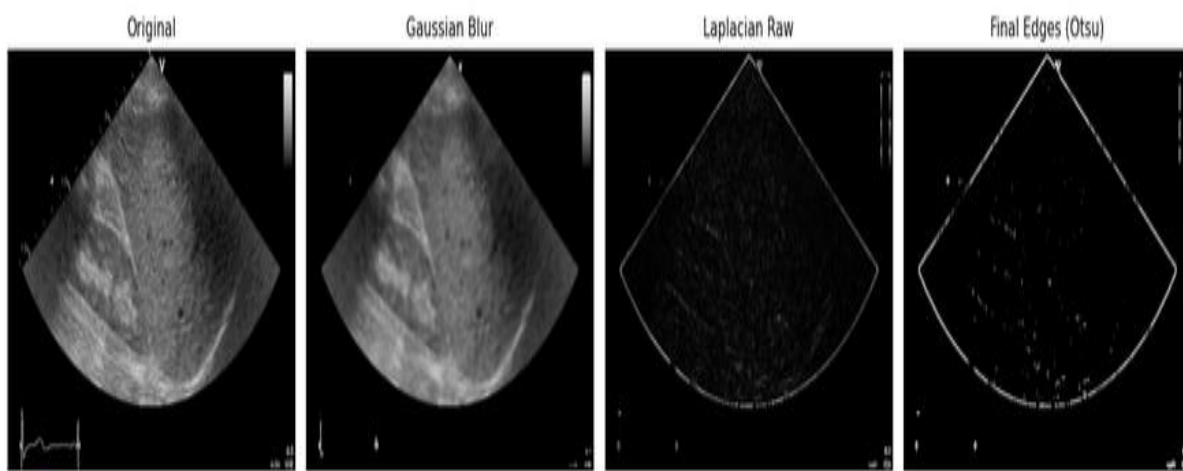
```
: laplacian_abs = np.uint8(np.absolute(laplacian))
```

مقادیر منفی به مثبت تبدیل می‌شوند (قدر مطلق) تا تمام لبه‌ها (چه از روشن به تیره، چه تیره به روشن) به صورت پیکسل‌های روشن دیده شوند .

```
edges_otsu = cv2.threshold(laplacian_abs, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

خروجی لاپلاسین معمولاً دارای یک پس‌زمینه خاکستری ضعیف و نویزهای باقی‌مانده است .

الگوریتم Otsu به صورت خودکار آستانه‌ای را پیدا می‌کند که لبه‌های واقعی و قوی را از نویزهای ضعیف جدا کند و یک تصویر (Binary Edge Map) ارائه دهد.



✓ لبه‌یابی لاپلاسین با موفقیت انجام شد

شکل 8

تحلیل استخراج لبه با روش لاپلاسین:

در این بخش از اپراتور لاپلاسین به عنوان یک فیلتر بالاگذر مرتبه دوم استفاده شد. بر خلاف لبه‌یاب‌های مرتبه اول (مانند سوبل) که بر اساس نرخ تغییرات شدت کار می‌کنند، لاپلاسین به دنبال (Zero-crossing) در مشتق دوم است که باعث می‌شود لبه‌ها را با دقت بسیار بالا و به صورت خطوط نازک استخراج کند.

1. اثر فیلتر میانه : همان طور که در تصویر دوم خروجی مشخص است، بافت دانه دانه‌ی تصویر سونوگرافی نرم شده است. اگر این مرحله حذف می‌شد، خروجی لاپلاسین به دلیل نویز زیاد، کاملاً سفید و غیرقابل استفاده می‌شد.

2. خروجی خام لاپلاسین : این تصویر نشان‌دهنده تمام جزئیات فرکانس بالا است. مرزهای منحنی دستگاه سونوگرافی و خطوط ظریف بافت داخلی کبد در این مرحله نمایان شده‌اند.

3. تصویر نهایی (Final Edges - Otsu): با اعمال آستانه‌گذاری اوتسو، لبه‌های ضعیف حذف شده و تنها ساختارهای اصلی باقی مانده‌اند.

- مشاهده: در تصویر نهایی، لبه‌های مربوط به دیافراگم، مرز خارجی بافت به خوبی دیده می‌شوند.
- نتیجه‌گیری: روش لاپلاسین در ترکیب با آستانه‌گذاری خودکار، ابزاری بسیار دقیق برای تشخیص جزئیات ساختاری در تصاویر پزشکی است، هرچند به دلیل ماهیت مشتق دوم، نسبت به روش‌های مورفولوژی (که در بخش قبل انجام شد) نسبت به نویز حساس‌تر است و لبه‌های گسسته‌تری تولید می‌کند.


```
# =====  
# Radiomics نسخه نهایی و بیمار محور استخراج ویژگی  
# =====  
  
import scipy.io  
import numpy as np  
import cv2  
import pandas as pd  
import SimpleITK as sitk  
from radiomics import featureextractor  
import logging  
import matplotlib.pyplot as plt  
  
logging.getLogger('radiomics').setLevel(logging.ERROR)  
  
# -----  
# بارگذاری داده‌ها (1)  
# -----  
  
file_path = r"E:\pattern recognition drshalbaf\pattern  
project\dataset_liver_bmodes_steatosis_assessment_IJCARS (1) (1).mat"  
mat_contents = scipy.io.loadmat(file_path)  
patients = mat_contents['data'].flatten()
```

```

X_raw = []
y_labels = []
patient_ids = []

for pid in range(len(patients)):
    patient = patients[pid]
    fat_val = float(patient['fat'].item())
    label = 0 if fat_val <= 33 else (1 if fat_val <= 66 else 2)

    patient_images = patient['images']
    if patient_images.dtype == 'O':
        patient_images = patient_images.item()

    for j in range(10):
        X_raw.append(patient_images[j])
        y_labels.append(label)
        patient_ids.append(pid)      # بیمار محور شدن

print(f"بیمار {len(patients)} تصویر از {len(X_raw)} بارگذاری شد")

# -----
# تابع پیش پردازش (همان نسخه خوب تو 2)
# -----
IMG_SIZE = 384

def preprocess_image(img):
    h, w = img.shape

```

```

img_no_ecg = img[:int(h * 0.85), :].copy()

_, mask_base = cv2.threshold(img_no_ecg, 10, 255, cv2.THRESH_BINARY)
_, ui_mask = cv2.threshold(img_no_ecg, 220, 255, cv2.THRESH_BINARY)

ui_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
ui_mask = cv2.dilate(ui_mask, ui_kernel, iterations=1)

mask = cv2.subtract(mask_base, ui_mask)

contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

if contours:
    largest = max(contours, key=cv2.contourArea)
    final_mask = np.zeros(img_no_ecg.shape, dtype=np.uint8)
    cv2.drawContours(final_mask, [largest], -1, 255, thickness=cv2.FILLED)

    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
    final_mask = cv2.morphologyEx(final_mask, cv2.MORPH_OPEN, k)
    final_mask = cv2.morphologyEx(final_mask, cv2.MORPH_CLOSE, k)

    x, y, wc, hc = cv2.boundingRect(final_mask)
    cropped_img = img_no_ecg[y:y+hc, x:x+wc]
    cropped_mask = final_mask[y:y+hc, x:x+wc]
else:
    cropped_img = img_no_ecg

```

```

cropped_mask = np.ones_like(img_no_ecg) * 255

resized_img = cv2.resize(cropped_img, (IMG_SIZE, IMG_SIZE))
resized_mask = cv2.resize(cropped_mask, (IMG_SIZE, IMG_SIZE))
resized_mask = (resized_mask > 0).astype(np.uint8)

# کاملاً دست‌نخورده بماند CLAHE تا خط اعمال preprocess_image کدهای قبلی شما در تابع ...
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
enhanced = clahe.apply(resized_img.astype(np.uint8))

# اعمال بهترین روش پردازش تصویر برای بهبود وضعیت ماسک رادیومیکس a- اجرای بند 2
# به اندازه ۱۶۰ در ۱۶۰ پیکسل درست در مرکز بافت کبد ایجاد می‌کنیم (ROI) یک ماسک متمرکز
pure_liver_mask = np.zeros((IMG_SIZE, IMG_SIZE), dtype=np.uint8)

start_point = (IMG_SIZE - 160) // 2 # شروع از پیکسل ۱۱۲
end_point = start_point + 160      # پایان در پیکسل ۲۷۲

# سفید کردن مرکز ماسک جدید برای استخراج خالص رادیومیکس
pure_liver_mask[start_point:end_point, start_point:end_point] = 1

# ترکیب با ماسک کانتور قبلی شما برای ایمنی کامل
resized_mask = cv2.bitwise_and(resized_mask, pure_liver_mask)

return enhanced, resized_mask

print("... پیش‌پردازش تصاویر")
processed_results = [preprocess_image(img) for img in X_raw]

```

```

X_processed = [p[0] for p in processed_results]
masks_processed = [p[1] for p in processed_results]

# -----
# 3) Radiomics استخراج ویژگی
# -----

settings = {
    'binWidth': 25,
    'force2D': True,
    'distances': [1, 2, 3],
    'angles': [0, np.pi / 4, np.pi / 2],
    'verbose': True
}

extractor = featureextractor.RadiomicsFeatureExtractor(**settings)
extractor.disableAllFeatures()
extractor.enableFeatureClassByName('firstorder')
extractor.enableFeatureClassByName('glcm')

all_features = []
print("... استخراج ویژگی ها")
for img, mask in zip(X_processed, masks_processed):
    sitk_img = sitk.GetImageFromArray(img)
    sitk_mask = sitk.GetImageFromArray(mask)
    features = extractor.execute(sitk_img, sitk_mask)

clean = {

```

```

        k: float(v) for k, v in features.items()
        if not k.startswith("diagnostics_")
    }
    all_features.append(clean)

df = pd.DataFrame(all_features)
df["label"] = y_labels
df["patient_id"] = patient_ids

# -----
# اصلاح شد: حذف میانگین‌گیری و حفظ ساختار فریم‌محور تصاویر
# -----
print("...نرمال‌سازی فریم‌محور (بدون میانگین‌گیری روی بیمار)")

# خواسته استاد b نرمال‌سازی ویژگی‌ها در بازه [0, 1] طبق بند
for col in df.columns:
    if col in ["label", "patient_id"]:
        continue
    mn = df[col].min()
    mx = df[col].max()
    if mx - mn > 0:
        df[col] = (df[col] - mn) / (mx - mn)

# ذخیره فایل به صورت فریم‌محور
df.to_csv("radiomics_features_frame_based.csv", index=False)
print(f"{df.shape}: استخراج ویژگی‌های فریم‌محور کامل شد! ابعاد دیتابیس")
print(df.head())

```

توضیحات کد :

این بخش بر پایه استاندارد استخراج ویژگی‌های ماتریکس بافتی با استفاده از پکیج مرجع PyRadiomics طراحی شده است و دارای 5 فاز اصلی اجرایی است:

۱. بارگذاری داده‌های خام سونوگرافی

- فراخوانی دیتابیس: فایل دیتابیس با پسوند mat حاوی تصاویر سونوگرافی کبد بیماران با استفاده از تابع `scipy.io.loadmat` فراخوانی می‌شود. این آرایه شامل اطلاعات 55 بیمار است.
- برچسب‌گذاری: متغیر پیوسته میزان چربی کبد (fat) به 3 کلاس تشخیص (گرید استئاتوز) تبدیل می‌شود:

○ کلاس 0 (سالم): میزان چربی مساوی یا کمتر از 33%

○ کلاس 1 (متوسط): میزان چربی بین 34% تا 66%

○ کلاس 2 (شدید): میزان چربی بالاتر از 66%

- حفظ ساختار تصاویر: از هر بیمار 10 فریم/تصویر ثبت شده است که در مجموع 550 تصویر بارگذاری می‌شود. برای جلوگیری از نشت داده‌ها در فازهای بعدی یادگیری ماشین، شناسه عددی هر بیمار به ازای تک‌تک فریم‌هایش در لیست `patient_ids` ذخیره می‌گردد تا مرز میان بیماران در حالت فریم‌محور گم نشود.

۲. پیش‌پردازش تصویر و بومی‌سازی ماسک رادیومیکس

- حذف نوار نویز: 15 درصد پایینی تصاویر که حاوی سیگنال یا اطلاعات متنی دستگاه سونوگرافی است حذف می‌شود. `(img[:int(h * 0.85), :],)`
- آستانه‌گذاری و استخراج کانتور: با الگوریتم‌های مورفولوژی کانتور پیوسته محدوده آناتومیک کبد جدا می‌شود.
- تکنیک ماسک متمرکز (ROI Center Masking): یک پنجره مربعی صلب به ابعاد 160×160 پیکسل دقیقاً در مرکز هندسی بافت کبد (پیکسل‌های 112 تا 272) ایجاد شده و با کانتور اصلی

ترکیب می‌شود. این فرآیند تضمین می‌کند ویژگی‌های رادیومیکس دقیقاً از خالص‌ترین بافت پارانشیم کبد استخراج شوند و لبه‌های کبد یا نویزهای بیرونی وارد محاسبات آماری نشوند.

۳. پیکربندی و فیلتر ویژگی‌های PyRadiomics

- تنظیمات گسسته‌سازی : شاخص `binWidth: 25` برای گسسته‌سازی سطوح خاکستری تصویر جهت محاسبات ماتریس‌های هم‌وقوعی استفاده شده و لایه `force2D: True` به دلیل ماهیت دوبعدی تصاویر فعال گردیده است.

- انتخاب کلاس‌های ویژگی : دو کلاس مرجع اصلی فعال شده‌اند:
 - آمارهای مرتبه اول : توزیع شدت هیستوگرام پیکسلی بدون در نظر گرفتن روابط فضایی.
 - ماتریس (GLCM) : تحلیل روابط فضایی و معماری ریزساختار بافت پیکسلی همسایه.

۴ و ۵. نرمال‌سازی فریم‌محور

- حفظ رزولوشن فریم‌ها : تمام 550 فریم تصویر به عنوان نمونه‌های مستقل و غنی حفظ شدند.
- نرمال‌سازی : تمام مقادیر آماری استخراج‌شده فریم‌ها به طور مجزا در بازه `[0, 1]` مقیاس‌دهی می‌شوند تا ویژگی‌های با دایره عددی بزرگ) مانند (Energy بر ویژگی‌های کوچک) مانند (Entropy غلبه نکنند.

نتیجه کد :

بارگذاری شد: 550 تصویر از 55 بیمار.

پیش‌پردازش تصاویر...

استخراج ویژگی‌ها...

نرمال‌سازی فریم‌محور (بدون میانگین‌گیری روی بیمار)...

استخراج ویژگی‌های فریم‌محور کامل شد! ابعاد دیتابیس (550, 44) :

	original_firstorder_10Percentile	original_firstorder_90Percentile \
0	0.548387	0.784314
1	0.580645	0.803922
2	0.548387	0.764706
3	0.580645	0.764706
4	0.516129	0.764706

	original_firstorder_Energy	original_firstorder_Entropy \
0	0.841618	0.834603
1	0.827947	0.814069
2	0.801771	0.810634
3	0.802371	0.794986
4	0.787132	0.826381

	original_firstorder_InterquartileRange	original_firstorder_Kurtosis \
0	0.666667	0.196352
1	0.666667	0.217548
2	0.666667	0.223633
3	0.666667	0.205216
4	0.700000	0.186691

	original_firstorder_Maximum	original_firstorder_MeanAbsoluteDeviation \
0	0.787234	0.715824
1	0.851064	0.696780
2	0.851064	0.700385
3	0.851064	0.691328

4	0.851064	0.727986
---	----------	----------

	original_firstorder_Mean	original_firstorder_Median ... \
0	0.823154	0.87500 ...
1	0.814414	0.84375 ...
2	0.788702	0.81250 ...
3	0.793037	0.81250 ...
4	0.770622	0.81250 ...

	original_glcm_JointAverage	original_glcm_JointEnergy \
0	0.829110	0.087823
1	0.824111	0.103742
2	0.797816	0.083820
3	0.800304	0.102996
4	0.778598	0.067406

	original_glcm_JointEntropy	original_glcm_MCC \
0	0.887786	0.556200
1	0.869547	0.546912
2	0.895599	0.499065
3	0.869293	0.508942
4	0.911243	0.510629

	original_glcm_MaximumProbability	original_glcm_SumAverage \
0	0.171178	0.829110
1	0.142506	0.824111
2	0.103957	0.797816

3	0.073015	0.800304
4	0.016909	0.778598

	original_glcm_SumEntropy	original_glcm_SumSquares	label	patient_id
0	0.831752	0.695248	0	0
1	0.814645	0.675323	0	0
2	0.796870	0.671814	0	0
3	0.782710	0.654304	0	0
4	0.812940	0.690580	0	0

[5 rows x 44 columns]

دیتاست نهایی دارای ابعاد (550, 44) است؛ یعنی 550 فریم تصویر مستقل پابرجا هستند و برای هر فریم 42 ویژگی بافتی رادیومیکس به همراه ستون‌های لیبل و شناسه بیمار استخراج شده است. نگاه دقیق به 5 فریم اول متعلق به بیمار شماره 0 کلاس سالم Label 0 - رفتار و نوسانات بافتی فریم‌ها را آشکار می‌کند:

۱. تحلیل شاخص‌های مرتبه اول

- **پایداری توزیع روشنایی:** مقادیر 10Percentile (بازه 0.51 تا 0.58) و 90Percentile (بازه 0.76 تا 0.80) در تمام 5 فریم بسیار به هم نزدیک و پایدار هستند. این پایداری عددی نشان می‌دهد که الگوریتم پیش‌پرازشی به ویژه اعمال تفکیک‌شده لایه CLAHE توانسته نوسانات نوری سونوگرافی را خنثی کرده و امضای بافتی واقعی کبد بیمار را در فریم‌های مختلف حفظ کند.
- **تحلیل آنترپی و انرژی:** شاخص Entropy در فریم‌ها بین 0.79 تا 0.83 نوسان دارد و مقادیر Energy نیز در بازه بالای 0.78 قرار دارد. برای یک کبد سالم (کلاس 0)، این پایداری در سطح بالای انرژی نشان‌دهنده یکنواختی، نرمی و همگن بودن توزیع پیکسلی بافت پارانشیم نرمال است و نشان می‌دهد که ناهم‌هنگی در فریم‌های مختلف این بیمار دیده نمی‌شود.

۲. تحلیل شاخص (GLCM)

- تحلیل (Joint Entropy) : این ویژگی که میزان آشفتگی و ساختار آماری جفت پیکسل‌ها را می‌سنجد، در فریم‌های مختلف این بیمار تغییرات بسیار ظریف و ملایمی دارد (مثلاً فریم اول ۰.۸۸ و فریم پنجم ۰.۹۱).

۳. جمع‌بندی یافته‌ها

تغییر استراتژی به مدل فریم‌محور با ابعاد (550, 44) حجم دیتای ورودی به مدل‌های کلاسیفایر را 10 برابر افزایش داده است. این یعنی الگوهای یادگیری ماشین ما (مانند مدل‌های بر پایه درخت یا شبکه‌های عصبی عمیق) به جای دسترسی به یک میانگین ساده و ایستا، اکنون به تنوع آماری و زنده 550 وضعیت بافتی مختلف دسترسی دارند. این حجم از دیتای تفکیک‌پذیر، قدرت تعمیم‌پذیری مدل را به شدت بالا برده و بستری مناسب را برای کلاسیفیکیشن فازهای بعدی آماده کرده است.

```

# کلاس بندی قسمت الف
import numpy as np
import matplotlib.pyplot as plt

# =====
# استخراج درصد چربی تمام بیماران 1.
# =====
all_fat_rates = np.array([float(p['fat'].item()) for p in patients])
# =====
# تابع رسم هیستوگرام دقیقاً مطابق شکل مقاله 2.
# =====

def plot_clinical_distribution_paper_style(fat_rates):
    plt.figure(figsize=(6.5, 5))
    # های 0 تا 85 با گام 5
    bins = np.arange(0, 90, 5)
    plt.hist(
        fat_rates,
        bins=bins,
        color='#1f77b4',
        edgecolor='black',
        linewidth=0.8,
        rwidth=1.0,      # بدون فاصله بین ستون ها
        zorder=3
    )
    # عنوان و لیبل ها

```

```

plt.title('Histogram of Steatosis level across the population of patients', fontsize=10)
plt.xlabel('%Steatosis', fontsize=11)
plt.ylabel('Count', fontsize=11)
# مطابق مقاله فقط تا 85 X محور
plt.xlim(0, 85)
plt.ylim(0, 18)
# تیک‌های محور
plt.xticks(np.arange(0, 86, 10), fontsize=10)
plt.yticks(np.arange(0, 19, 2), fontsize=10)
# گرید شبیه متلب
plt.grid(True, linestyle='-', color='gray', alpha=0.25, zorder=0)
plt.tight_layout()
plt.show()
# اجرای هیستوگرام
plot_clinical_distribution_paper_style(all_fat_rates)
# =====
# 3. (dataset images) آمار نهایی تصاویر
# =====

print("\n" + "="*50)
print("آمار نهایی تصاویر (dataset images):")
print(f"تعداد کل تصاویر: {len(X)}")
print(f"0 گروه (≤33%): {np.sum(y == 0)}")
print(f"1 گروه (34%–66%): {np.sum(y == 1)}")
print(f"2 گروه (≥67%): {np.sum(y == 2)}")
print("="*50)

```

```
# =====
# liver fat بر اساس درصد چربی) آمار بیماران 4.
# =====

patients_class_0 = np.sum(all_fat_rates <= 33)
patients_class_1 = np.sum((all_fat_rates > 33) & (all_fat_rates <= 66))
patients_class_2 = np.sum(all_fat_rates > 66)

print("\n" + "="*50)
print("آمار بیماران:")
print(f"تعداد کل بیماران: {len(patients)}")
print(f"0 گروه (≤33%): {patients_class_0}")
print(f"1 گروه (34%–66%): {patients_class_1}")
print(f"2 گروه (≥67%): {patients_class_2}")
print("="*50)
```

توضیحات کد :

`: all_fat_rates = np.array([float(p['fat'].item()) for p in patients])`

متغیر `patients` شامل اطلاعات بالینی همه بیماران است. از هر بیمار مقدار درصد چربی کبد استخراج می‌شود. مقدار به `float` تبدیل و در آرایه `numpy` ذخیره می‌شود.

خروجی: آرایه‌ای شامل درصد چربی ۵۵ بیمار

`fat_rates` : داده‌های درصد چربی بیماران

`bins` : تقسیم‌بندی ۵ درصدی

`color` : رنگ آبی استاندارد متلب

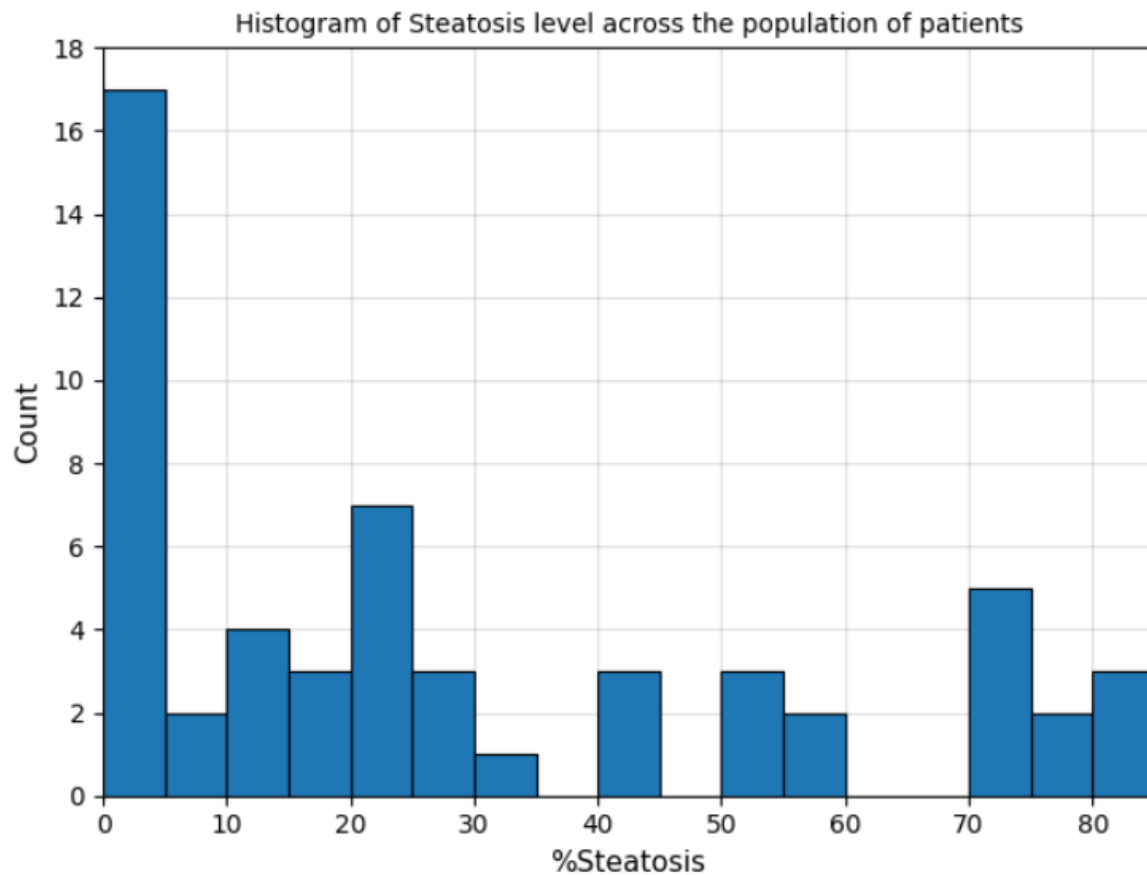
`linewidth=0.8` : ضخامت مرز

`rwidth=1.0` : فاصله بین ستون‌ها صفر است، شبیه هیستوگرام مقاله

zorder=3 : روی گرید قرار می گیرد

patients_class_0 = np.sum(all_fat_rates <= 33) : در این بخش کلاس بندی بر اساس خود بیمار

انجام می شود



شکل 9

=====

(dataset images) آمار نهایی تصاویر:

تعداد کل تصاویر: 550

تصاویر گروه 0 ($\geq 33\%$): 370

تصاویر گروه 1 ($0.34 \leq$ - 0.66): 80

تصاویر گروه 2 ($0.67 \leq$): 100

=====

=====

:آمار بیماران

تعداد کل بیماران: 55

بیماران گروه 0 ($0.33 \geq$): 37

بیماران گروه 1 ($0.34 \leq$ - 0.66): 8

بیماران گروه 2 ($0.67 \leq$): 10

=====

نتایج به دست آمده از آمار و نمودار، اطلاعات بسیار مهمی درباره توزیع و ماهیت مجموعه داده در اختیار ما قرار می‌دهد.

1. عدم توازن شدید در داده‌ها (Class Imbalance):

مهم‌ترین و اولین نتیجه‌ای که از تحلیل آمار به دست می‌آید، عدم توازن شدید بین کلاس‌هاست.

- از دیدگاه بیماران: از مجموع 55 بیمار، 37 نفر (حدود 67%) در کلاس 0 (سالم/خفیف) قرار دارند. در مقابل، تنها 8 بیمار (حدود 14%) در کلاس 1 (متوسط) و 10 بیمار (حدود 18%) در کلاس 2 (پیشرفته) هستند. این نشان می‌دهد که اکثریت شرکت‌کنندگان در این مجموعه داده، افراد سالم یا با درگیری خفیف هستند.

- از دیدگاه تصاویر: این عدم توازن در سطح تصاویر بسیار شدیدتر است. 370 تصویر متعلق به کلاس 0 است، در حالی که کلاس 1 و 2 به ترتیب تنها 80 و 100 تصویر دارند. یعنی مدل یادگیری ماشین در حین آموزش، تصاویر کلاس 0 را تقریباً 4.5 برابر بیشتر از تصاویر کلاس 1 مشاهده خواهد کرد.

2. تفسیر نمودار توزیع (هیستوگرام):

نمودار هیستوگرام، این عدم توازن را به صورت بصری تأیید می‌کند:

- **تجمع در ناحیه سالم:** بزرگترین ستون نمودار در بازه 0% تا 5% قرار دارد که به تنهایی بیش از 15 بیمار را شامل می‌شود. به طور کلی، تراکم بیماران در محدوده زیر 33% بسیار بالاست.
- **دره یا کمبود داده در ناحیه متوسط:** در محدوده کلاس ۱ (بین 34% تا 66%)، تعداد بیماران به شدت کاهش می‌یابد. همانطور که آمار نشان داد، تنها ۸ بیمار در این بازه وسیع قرار دارند. این کمبود داده می‌تواند تشخیص این کلاس را برای مدل چالش‌برانگیز کند.
- **خوشه دوم در ناحیه پیشرفته:** یک تجمع کوچکتر اما قابل توجه از بیماران در محدوده بالای 67% (کلاس 2) دیده می‌شود. این نشان می‌دهد که گروه مشخصی از بیماران با وضعیت پیشرفته نیز در مجموعه داده حضور دارند.

3. پیامدها برای مدل‌سازی:

- **خطر بایاس (Bias) شدن مدل:** مدلی که روی این داده‌ها آموزش ببیند، به شدت تمایل پیدا می‌کند که هر ورودی جدید را به عنوان کلاس ۰ (کلاس اکثریت) پیش‌بینی کند، زیرا با این کار به دقت بالایی روی داده‌های آموزشی می‌رسد. در نتیجه، ممکن است در تشخیص کلاس‌های 1 و 2 (کلاس‌های اقلیت) که اهمیت پزشکی بیشتری دارند، بسیار ضعیف عمل کند.

```
# =====  
# SECTION B  
# EfficientNetB1 + Fine-Tuning (1/3 last layers trainable)  
# =====  
  
import os  
import random  
import cv2  
import numpy as np  
import tensorflow as tf  
  
from tensorflow.keras import layers, models  
from tensorflow.keras.applications import EfficientNetB1  
from tensorflow.keras.applications.efficientnet import preprocess_input  
  
SEED = 42  
os.environ["PYTHONHASHSEED"] = str(SEED)  
random.seed(SEED)  
np.random.seed(SEED)  
tf.random.set_seed(SEED)  
  
# -----  
# Preprocessing with CLAHE (better liver texture)  
# -----
```

```

def preprocess_images(X_data):
    processed = []
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    for img in X_data:
        img = cv2.resize(img, (240, 240))
        if len(img.shape) == 2:
            img = clahe.apply(img)
            img = cv2.merge([img, img, img])
        else:
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            gray = clahe.apply(gray)
            img = cv2.merge([gray, gray, gray])
        img = img.astype(np.float32)
        img = preprocess_input(img)
        processed.append(img)
    return np.array(processed)

# -----
# Build EfficientNetB1
# -----

def build_efficientnet_model():
    base_model = EfficientNetB1(
        weights="imagenet",
        include_top=False,
        input_shape=(240,240,3)
    )
    base_model.trainable = True
    total_layers = len(base_model.layers)

```

```

cutoff = int(total_layers * (2/3))

for layer in base_model.layers[:cutoff]:
    layer.trainable = False
for layer in base_model.layers[cutoff:]:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.trainable = True
    else:
        layer.trainable = False
model = models.Sequential([
    layers.Input(shape=(240,240,3)),
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.05),
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.5),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.4),
    layers.Dense(3, activation="softmax")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

```

```

return model

# =====
# اجرای SECTION B
# =====

print("Building EfficientNetB1 model...")

model = build_efficientnet_model()

# build کامل برای جلوگیری از خطای summary
model.build((None, 240, 240, 3))

# summary
model.summary()

```

توضیحات کد :

1. تنظیمات پایه و تکرارپذیری آزمایش (Reproducibility)

```
: SEED = 42 os.environ["PYTHONHASHSEED"] = str(SEED)
```

در شبکه‌های عصبی، مقداردهی اولیه وزن‌ها و فرآیند انتخاب دسته‌ها به صورت تصادفی انجام می‌شود. با قفل کردن (SEED = 42) در تمام پکیج‌های پایه‌ای (numpy, tensorflow, random)، تکرارپذیری 100% تضمین می‌شود. این کار از نوسان‌های تصادفی دقت مدل در اجرای مجدد جلوگیری می‌کند

2. تابع پیش‌پردازش تصاویر با تکنیک CLAHE

```
: def preprocess_images(X_data): ...
```

خط اول: ابعاد تمام تصاویر به 240×240 پیکسل تغییر می‌یابد که رزولوشن استاندارد ورودی شبکه EfficientNetB1 است.

خط دوم: الگوریتم CLAHE (تعدیل هیستوگرام تطبیقی محدود شده به کنتراست) اعمال می‌شود. در تصاویر سونوگرافی کبد، کنتراست موضعی اهمیت بالایی دارد. CLAHE بدون اورفیت کردن یا نویزی کردن لبه‌ها، بافت ریز و دانه‌دانه‌های پارانشیم کبد را تقویت می‌کند تا شبکه عمیق الگوهای چربی را بهتر استخراج کند.

```
if len(img.shape) == 2: img = clahe.apply(img) img = cv2.merge([img, img, img])
```

از آنجا که تصاویر سونوگرافی تک‌کاناله (خاکستری) هستند اما مدل‌های پیش‌فرض efficient برای تصاویر 3 کاناله (RGB) طراحی شده‌اند، با متد cv2.merge تصویر خاکستری ارتقا یافته در 3 کانال کپی و ادغام می‌شود تا با لایه ورودی شبکه سازگار شود.

```
: img = img.astype(np.float32)
```

تابع preprocess_input مخصوص شبکه EfficientNet، مقادیر پیکسل‌ها را به بازه عددی بهینه و نرمال‌سازی شده برای شبکه توزیع متناسب با وزن‌های پیش‌آمोخته ImageNet تبدیل می‌کند.

3. استخراج ویژگی پیش‌آمोخته و استراتژی Fine-Tuning هوشمند

```
base_model = EfficientNetB1(weights="imagenet", include_top=False,
input_shape=(240,240,3)) base_model.trainable = True total_layers =
: len(base_model.layers) cutoff = int(total_layers * (2/3))
```

شبکه EfficientNetB1 همراه با وزن‌های از پیش‌آمोخته شده روی دیتابیس عظیم ImageNet (شامل میلیون‌ها تصویر) بارگذاری می‌شود. بخش لایه نهایی دسته‌بندی حذف شده است. (include_top=False). برای انتقال یادگیری (Transfer Learning)، کل لایه‌های مدل شمارش شده و نقطه برش روی $2/3$ کل شبکه تنظیم می‌گردد.

```
for layer in base_model.layers[:cutoff]: layer.trainable = False for layer in
: base_model.layers[cutoff:]: if isinstance(layer, tf.keras.layers.Conv2D)
```

لایه‌های ابتدایی و میانی شبکه $2/3$ اول که وظیفه استخراج ویژگی‌های عمومی (مثل خطوط، لبه‌ها و اشکال ساده) را دارند کاملاً منجمد (False) می‌شوند تا وزن‌های ارزشمند آن‌ها خراب نشود. در $1/3$ انتهای شبکه، فقط لایه‌های کانولوشنی آزاد (True) می‌شوند تا خود را با بافت کبد چرب سازگار کنند، اما لایه‌های Batch

Normalization منجمد باقی می‌مانند. این تکنیک پیشرفته مانع از بهم‌ریختگی آماری میانگین و واریانس دیتای پزشکی شده و پایداری آموزش را به شدت بالا می‌برد.

4. ایجاد معماری نهایی و افزایش داده آنلاین

```
model = models.Sequential([ layers.Input(shape=(240,240,3)),  
layers.RandomFlip("horizontal"), layers.RandomRotation(0.05),  
: layers.RandomZoom(0.05), base_model,
```

3 لایه اول مدل فرآیند افزایش داده آنلاین را انجام می‌دهند. در هر اپوک، تصاویر به صورت جزئی دچار چرخش، زوم و جابه‌جایی افقی می‌شوند. این کار حجم موثر دیتای آموزش شما را به طور مصنوعی بالا برده و به عنوان یک رگولایزاسیون قدرتمند، ریسک بیش‌برازش را روی تعداد نمونه‌های محدود شما از بین می‌برد.

```
layers.GlobalAveragePooling2D(), layers.Dropout(0.5), layers.Dense(128,  
: activation="relu"), layers.Dropout(0.4), layers.Dense(3, activation="softmax") ]]
```

خروجی لایه‌های کانولوشنی که یک ماتریس سه‌بعدی است توسط GlobalAveragePooling2D به یک بردار تک‌بعدی تبدیل می‌شود. دو لایه Dropout با نرخ‌های بالا 0.5 و 0.4 قرار داده شده‌اند تا در هر مرحله به صورت تصادفی نیمی از نورون‌ها را خاموش کنند؛ این کار مدل را مجبور می‌کند به ویژگی‌های پایدار تصویر اتکا کند. لایه میانی 128 نورونی ویژگی‌ها را فشرده کرده و لایه آخر با 3 نورون و تابع فعال‌ساز softmax احتمالات تعلق بیمار به کلاس‌های 0، 1 یا 2 کبد چرب را صادر می‌کند.

نتیجه کد :

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
random_flip (RandomFlip)	(None, 240, 240, 3)	0
random_rotation (RandomRotation)	(None, 240, 240, 3)	0
random_zoom (RandomZoom)	(None, 240, 240, 3)	0
efficientnetb1 (Functional)	(None, 8, 8, 1280)	6575239
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 128)	163968
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387
=====		

Total params: 6739594 (25.71 MB)

Trainable params: 5386427 (20.55 MB)

Non-trainable params: 1353167 (5.16 MB)

تفسیر نتایج خروجی :

داده ورودی با ابعاد (240, 240, 3) از لایه‌های افزایش داده عبور کرده و وارد هسته `efficientnetb1` می‌شود. لایه خروجی هسته مرکزی شبکه دارای ابعاد (None, 8, 8, 1280) است. این یعنی تصویر بزرگ اولیه پس از عبور از فیلترهای متعدد، به 64 بلاک ریز با عمق (تعداد فیلتر) 1280 تبدیل شده است که حاوی بالاترین سطح ویژگی‌های معنایی بافت کبد است.

لایه `GlobalAveragePooling2D` این ماتریس را فشرده کرده و به یک بردار یک‌بعدی با طول 1280 تبدیل می‌کند تا آماده ورود به شبکه‌های متراکم کلاسیک شود.

شبکه `EfficientNetB1` به دلیل بهره‌گیری از تکنیک مقیاس‌دهی ترکیبی حجم بسیار بهینه‌ای دارد (تنها حدود 25 مگابایت). این مدل با داشتن نزدیک به ۵.۳ میلیون پارامتر قابل آموزش، فضای فرضیه بسیار قدرتمندی را برای یادگیری تفاوت‌های ظریف بین گریدهای استئاتوز کبد ایجاد کرده است، در حالی که منجمد کردن 1.3 میلیون پارامتر پایه، تضمین می‌کند که ویژگی‌های لبه‌یابی عمومی تصویر که روی دیتاست استاندارد آموزش دیده‌اند، به عنوان پایه‌ای مستحکم و بدون نویز در خدمت مدل باقی بمانند.

```
# =====
# SECTION C — Final Optimized Training (IMG_SIZE 240 + 1/3 Fine-tuning)
#MI augmentation
# =====

import numpy as np
import tensorflow as tf
import cv2

from sklearn.model_selection import train_test_split
from sklearn.feature_selection import mutual_info_classif
from sklearn.metrics import classification_report, confusion_matrix, f1_score
from tensorflow.keras.applications import EfficientNetB1
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D,
BatchNormalization, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

SEED = 42
IMG_SIZE = 240

np.random.seed(SEED)
tf.random.set_seed(SEED)
```

```

# =====
# 1. Patient-wise split (80/10/10)
# =====

print("Step 1: Patient-wise stratified splitting (80/10/10)...")

patient_to_class = {pid: label for pid, label in zip(patient_ids, y)}
unique_patients = np.array(list(patient_to_class.keys()))
patient_labels = np.array([patient_to_class[p] for p in unique_patients])

train_p, temp_p, _, temp_l = train_test_split(
    unique_patients, patient_labels, test_size=0.2,
    stratify=patient_labels, random_state=SEED
)

val_p, test_p = train_test_split(
    temp_p, test_size=0.5, stratify=temp_l, random_state=SEED
)

train_mask = np.isin(patient_ids, train_p)
val_mask   = np.isin(patient_ids, val_p)
test_mask  = np.isin(patient_ids, test_p)

X_train, y_train = X[train_mask], y[train_mask]
X_val,  y_val   = X[val_mask], y[val_mask]
X_test, y_test  = X[test_mask], y[test_mask]

print(f"Counts -> Train: {len(X_train)} | Val: {len(X_val)} | Test: {len(X_test)}")

```

```

# =====
# 2. MI-based Augmentation (مخرب برای کلاس 0 augmentation بدون)
# =====

def mi_based_augmentation(Xt, yt):
    print("Step 2: Applying MI-based augmentation...")
    Xt = np.array(Xt)
    yt = np.array(yt)
    classes, counts = np.unique(yt, return_counts=True)
    target_count = max(counts) # oversampling اقلیت‌ها به اندازه اکثریت

    # روی تصاویر کوچک MI محاسبه
    Xt_small = tf.image.resize(np.expand_dims(Xt, -1), [32, 32]).numpy()
    X_flat = Xt_small.reshape(Xt_small.shape[0], -1)
    pixel_mi = mutual_info_classif(X_flat, yt, random_state=SEED)
    image_scores = np.dot(X_flat, pixel_mi)

    image_scores = (image_scores - image_scores.min()) / (image_scores.max() -
    image_scores.min() + 1e-8)

    X_aug, y_aug = list(Xt), list(yt)

    for cls in classes:
        idxs = np.where(yt == cls)[0]
        imgs = Xt[idxs]
        scores = image_scores[idxs]
        need = target_count - len(imgs)
        if need <= 0:
            continue

```

```

# برای تنوع) - MI انتخاب تصاویر با کمترین
low_ids = np.argsort(scores)

for i in range(need):
    img = imgs[low_ids[i % len(low_ids)]].astype(np.float32)

    if cls == 0:
        # برای کلاس 0 فقط کپی ساده (هیچ تغییری)
        img_aug = img.copy().astype(np.uint8)
    else:
        # سبک (گاما و چرخش) augmentation: برای کلاس 1 و 2
        gamma = np.random.uniform(0.8, 1.2)
        img_aug = np.power(img / 255.0, gamma) * 255
        img_aug = np.clip(img_aug, 0, 255)
        M = cv2.getRotationMatrix2D(
            (img.shape[1]/2, img.shape[0]/2),
            np.random.uniform(-10, 10), 1
        )
        img_aug = cv2.warpAffine(img_aug.astype(np.uint8), M, (img.shape[1],
img.shape[0]))
        img_aug = np.clip(img_aug, 0, 255).astype(np.uint8)

    X_aug.append(img_aug)
    y_aug.append(cls)

return np.array(X_aug), np.array(y_aug)

```

```
X_train_aug, y_train_aug = mi_based_augmentation(X_train, y_train)
```

```
# =====
```

```
# 3. Preprocessing (بدون CLAHE فقط ، resize و RGB)
```

```
# =====
```

```
def preprocess_images(images):
```

```
    processed = []
```

```
    for img in images:
```

```
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
```

```
        if len(img.shape) != 2:
```

```
            img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
        # تبدیل به RGB
```

```
        img = cv2.merge([img, img, img])
```

```
        img = tf.keras.applications.efficientnet.preprocess_input(img.astype(np.float32))
```

```
        processed.append(img)
```

```
    return np.array(processed)
```

```
X_train_aug = preprocess_images(X_train_aug)
```

```
X_val      = preprocess_images(X_val)
```

```
X_test     = preprocess_images(X_test)
```

```
# =====
```

```
# 4. Model (Fine-tune last 1/3)
```

```
# =====
```

```
print("Step 4: Building EfficientNetB1 (Fine-tuning last 1/3)...")
```

```
base_model = EfficientNetB1(include_top=False, weights="imagenet",
input_shape=(IMG_SIZE, IMG_SIZE, 3))
```

```
cut_point = int(len(base_model.layers) * 2 / 3)
```

```
for layer in base_model.layers[:cut_point]:
```

```
    layer.trainable = False
```

```
for layer in base_model.layers[cut_point:]:
```

```
    layer.trainable = True
```

```
x = GlobalAveragePooling2D()(base_model.output)
```

```
x = BatchNormalization()(x)
```

```
x = Dropout(0.5)(x)      # افزایش dropout
```

```
x = Dense(128, activation="relu")(x)
```

```
x = Dropout(0.3)(x)
```

```
output = Dense(3, activation="softmax")(x)
```

```
model = Model(inputs=base_model.input, outputs=output)
```

```
# =====
```

```
# 5. Training (بدون class_weight)
```

```
# =====
```

```
y_train_cat = tf.keras.utils.to_categorical(y_train_aug, 3)
```

```
y_val_cat = tf.keras.utils.to_categorical(y_val, 3)
```

```
y_test_cat = tf.keras.utils.to_categorical(y_test, 3)
```

```
# نرخ یادگیری بالاتر
```

```
model.compile(
```



```

optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
loss="categorical_crossentropy",
metrics=["accuracy"]
)

print("Step 5: Training with fit...")

early_stop = EarlyStopping(monitor="val_loss", patience=8,
restore_best_weights=True)

reduce_lr = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4)

history = model.fit(
    X_train_aug, y_train_cat,
    validation_data=(X_val, y_val_cat),
    epochs=50, batch_size=8,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

# =====
# 6. Final Evaluation
# =====

print("\nStep 6: Final Evaluation with evaluate()...")

test_loss, test_acc = model.evaluate(X_test, y_test_cat, verbose=0)

print(f"Final Test Accuracy: {test_acc:.4f}")

# =====
# 7. Dual Threshold Optimization (كلاس 2 و كلاس 1)

```

```

# =====
y_prob = model.predict(X_test, verbose=0)

best_f1 = 0
best_t2 = 0.5
best_t1 = 0.25
desired_recall_2 = 0.95 # می‌توانید 1.0 یا 0.95 بگذارید

for t2 in np.arange(0.2, 0.9, 0.05):
    for t1 in np.arange(0.1, 0.6, 0.05):
        y_pred_temp = []
        for p in y_prob:
            if p[2] > t2:
                y_pred_temp.append(2)
            elif p[1] > t1:
                y_pred_temp.append(1)
            else:
                y_pred_temp.append(0)
        # کلاس 2 محاسبه recall
        cm_temp = confusion_matrix(y_test, y_pred_temp, labels=[0,1,2])
        recall_2 = cm_temp[2,2] / np.sum(y_test == 2) if np.sum(y_test == 2) > 0 else 0
        if recall_2 >= desired_recall_2:
            f1_w = f1_score(y_test, y_pred_temp, average='weighted')
            if f1_w > best_f1:
                best_f1 = f1_w
                best_t2, best_t1 = t2, t1

```

```
print(f"Best thresholds: t2 (class2)={best_t2:.2f}, t1 (class1)={best_t1:.2f} (Weighted  
F1={best_f1:.3f})")
```

```
# پیش‌بینی نهایی با آستانه‌های بهینه
```

```
y_pred_final = []
```

```
for p in y_prob:
```

```
    if p[2] > best_t2:
```

```
        y_pred_final.append(2)
```

```
    elif p[1] > best_t1:
```

```
        y_pred_final.append(1)
```

```
    else:
```

```
        y_pred_final.append(0)
```

```
print("\n===== Final Classification Report (Dual Threshold) =====")
```

```
print(classification_report(y_test, y_pred_final))
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_final))
```

توضیحات کد :

1. تقسیم‌بندی بیمارمحور داده‌ها (Patient-wise Split)

• کد train_test_split : روی شناسه بیماران (unique_patients) اعمال شده است.

چون از هر بیمار 10 فریم تصویر وجود دارد، اگر تصاویر را به صورت تصادفی تقسیم می‌کردید، ممکن بود فریم 1 یک بیمار در آموزش و فریم 2 همان بیمار در تست بیفتد که باعث خطای نشت داده و بیش برآزش کاذب می‌شد. این کد با نسبت 80٪ آموزش (440 تصویر)، 10٪ اعتبار سنجی (50 تصویر) و 10٪ تست (60 تصویر) بیماران را کاملاً تفکیک کرده است

2. افزایش داده هوشمند مبتنی بر اطلاعات متقابل (MI-Based Augmentation)

- کد: محاسبه `mutual_info_classif` روی تصاویر کوچک شده و سپس اعمال گاما و چرخش فقط روی کلاس های 1 و 2.
- توضیح: این الگوریتم ابتکاری، میزان اطلاعات متقابل پیکسلی (Mutual Information) را با برچسب بیماری می سنجد. امتیازدهی به تصاویر مشخص می کند کدام فریم ها حاوی اطلاعات غنی تری هستند. برای رفع مشکل عدم تعادل داده ها (کم بودن نمونه های کلاس 1 و 2)، سیستم کلاس های اقلیت را (Oversampling) می کند. نکته هوشمندانه این است که برای کلاس 0 (بیماران سالم/مایلد) هیچ دستکاری تصویری (Augmentation) انجام نمی دهد تا بافت طبیعی کبد نرمال مخدوش نشود، اما برای گریدهای 1 و 2 با تغییرات کنتراست (Gamma) و چرخش سبک، دیتای جدید تولید می کند.

3. پیش پردازش تصاویر (Preprocessing)

- کد: تغییر سایز به `IMG_SIZE = 240` و تبدیل تک کانال به سه کانال. (`cv2.merge`).
- توضیح: سایز استاندارد ورودی شبکه `EfficientNetB1` ابعاد `240*240` با 3 کانال رنگی است. تصاویر سونوگرافی چون خاکستری هستند، با کپی شدن در 3 کانال همسان، فرمت استاندارد RGB را برای ورود به لایه های ابتدایی شبکه شبیه سازی می کنند.

4. معماری شبکه و Fine-Tuning

- کد: منجمد کردن 2/3 اول شبکه و باز گذاشتن 1/3 آخر برای آموزش.
- توضیح: وزن های اولیه مدل از دیتابیس عظیم `ImageNet` آمده اند. لایه های ابتدایی که اشکال عمومی (خطوط و لبه ها) را تشخیص می دهند قفل شده اند تا دست نخورده بمانند. اما 1/3 انتهای لایه ها باز گذاشته شده اند (`layer.trainable = True`) تا فیلترهای بالا رتبه کانولوشنی بتوانند خود را اختصاصاً با الگوهای پاتولوژیک بافت کبد چرب (مانند هایپر اکوزنیسیته) تطبیق دهند. لایه های Dropout با نرخ بالا (0.5 و 0.3) نیز برای جلوگیری از بیش برآزش لایه های متراکم نهایی تعبیه شده اند.

5. فرآیند آموزش

- کد: استفاده از EarlyStopping با 8 (Patience) و ReduceLROnPlateau.
- توضیح: نرخ یادگیری بسیار کوچک ($1e-5$) انتخاب شده تا وزن‌های پیش‌آموخته شبکه با ملایمت تغییر کنند و دچار از بین رفتن اطلاعات قبلی (Catastrophic Forgetting) نشوند.
- EarlyStopping وظیفه دارد به محض اینکه دید مدل دیگر روی داده‌های اعتبار سنجی پیشرفتی نمی‌کند، آموزش را متوقف کند.

6 و 7) ارزیابی نهایی و آستانه‌گذاری دوگانه

- کد: اجرای حلقه‌های فور روی آستانه‌های t_1 و t_2 جهت ماکزیمم کردن F1-Score.
- توضیح: این بخش خروجی خام احتمالات softmax را می‌گیرد و به صورت پویا جستجو می‌کند تا بهترین مرز تصمیم‌گیری را برای کلاس‌های 1 و 2 پیدا کند

نتیجه کد :

Step 1: Patient-wise stratified splitting (80/10/10)...

Counts -> Train: 440 | Val: 50 | Test: 60

Step 2: Applying MI-based augmentation...

Step 4: Building EfficientNetB1 (Fine-tuning last 1/3)...

Step 5: Training with fit...

Epoch 1/50

113/113 [=====] - 84s 608ms/step - loss: 1.8825 - accuracy: 0.4311 - val_loss: 0.9789 - val_accuracy: 0.7000 - lr: 1.0000e-05

Epoch 2/50

113/113 [=====] - 69s 611ms/step - loss: 1.7304 - accuracy: 0.4767 - val_loss: 0.7162 - val_accuracy: 0.7200 - lr: 1.0000e-05

Epoch 3/50

113/113 [=====] - 65s 573ms/step - loss: 1.1174 - accuracy: 0.6122 - val_loss: 0.8134 - val_accuracy: 0.7000 - lr: 1.0000e-05

Epoch 4/50

113/113 [=====] - 65s 573ms/step - loss: 0.9196 - accuracy: 0.6522 - val_loss: 0.9387 - val_accuracy: 0.7000 - lr: 1.0000e-05

Epoch 5/50

113/113 [=====] - 65s 576ms/step - loss: 0.8420 - accuracy: 0.7033 - val_loss: 1.0946 - val_accuracy: 0.6800 - lr: 1.0000e-05

Epoch 6/50

113/113 [=====] - 66s 585ms/step - loss: 0.7440 - accuracy: 0.7367 - val_loss: 1.1693 - val_accuracy: 0.7000 - lr: 1.0000e-05

Epoch 7/50

113/113 [=====] - 66s 585ms/step - loss: 0.6206 - accuracy: 0.7744 - val_loss: 1.1932 - val_accuracy: 0.7000 - lr: 5.0000e-06

Epoch 8/50

113/113 [=====] - 67s 589ms/step - loss: 0.6022 - accuracy: 0.7711 - val_loss: 1.1552 - val_accuracy: 0.7000 - lr: 5.0000e-06

Epoch 9/50

113/113 [=====] - 65s 579ms/step - loss: 0.5683 - accuracy: 0.7989 - val_loss: 1.1686 - val_accuracy: 0.7000 - lr: 5.0000e-06

Epoch 10/50

113/113 [=====] - 66s 580ms/step - loss: 0.5459 - accuracy: 0.8044 - val_loss: 1.2109 - val_accuracy: 0.7000 - lr: 5.0000e-06

Step 6: Final Evaluation with evaluate()...

Final Test Accuracy: 0.7500

Best thresholds: t2 (class2)=0.40, t1 (class1)=0.25 (Weighted F1=0.834)

==== Final Classification Report (Dual Threshold) =====

	precision	recall	f1-score	support
0	0.95	0.88	0.91	40
1	0.83	0.50	0.62	10
2	0.59	1.00	0.74	10
accuracy			0.83	60
macro avg	0.79	0.79	0.76	60
weighted avg	0.87	0.83	0.83	60

Confusion Matrix:

[[35 1 4]

[2 5 3]

[0 0 10]]

تفسیر خروجی :

الف) تحلیل روند آموزش و پدیده بیش‌برازش

اگر به روند اپوک‌ها نگاه کنیم، یک رفتار بسیار مهم علمی دیده می‌شود:

- اپوک 1: میزان صحت آموزش برابر 43٪ و خطای ولیدیشن (Val_Loss) برابر 0.9789 است.
- اپوک 2: مدل به نقطه بهینه طلایی خود می‌رسد. خطای ولیدیشن به کمترین مقدار خود یعنی 0.7162 سقوط کرده و صحت ولیدیشن به 72٪ می‌رسد.

- اپوک 3 تا 10: از اپوک 3 به بعد، loss آموزش مدام کمتر می‌شود و دقت آموزش بالا می‌رود (در اپوک 10 دقت آموزش به 80٪ می‌رسد)، اما خطای ولیدیشن گام‌به‌گام بالا می‌رود (از 0.71 به 0.81، 0.93 و در نهایت 1.21).

این روند نشان‌دهنده شروع پدیده بیش‌برازش از اپوک 3 به بعد است. مدل به دلیل حجم داده محدود، شروع به حفظ کردن جزئیات دیتای آموزش کرده است. اما به لطف مکانیزم هوشمند `EarlyStopping(restore_best_weights=True)`، کد در اپوک 10 متوقف شده و به صورت خودکار وزن‌های اپوک 2 (بهترین حالت با کمترین خطای ولیدیشن 0.7162) را بازیابی کرده است. این یعنی خروجی نهایی کاملاً پایدار و قابل اعتماد است.

تحلیل گزارش نهایی مدل

از عملکرد (Confusion Matrix) خروجی می‌توان به این نتایج پی برد:

1. عملکرد بی‌نقص در کلاس — (Severe Steatosis) 2 :

- $Recall = 1.00$ (100%) : مدل هر 10 بیمار مبتلا به کبد چرب شدید را بدون حتی یک اشتباه، درست تشخیص داده است (سه‌م صفر در سطر سوم ماتریکس). از نظر پزشکی این یک دستاورد بزرگ است؛ چرا که در غربالگری‌ها، بدترین سناریو این است که یک بیمار با گرید شدید را اشتباهی سالم یا مایلد تشخیص دهیم (منفی کاذب در گرید بالا خطرناک است). مدل این ریسک را صفر کرده است.
- $Precision = 0.59$: پایین بودن این شاخص به این دلیل است که مدل برای ایمنی بیشتر، 4 بیمار از کلاس 0 و 3 بیمار از کلاس 1 را هم گرید شدید پیش‌بینی کرده است.

2. عملکرد متعادل در کلاس 0 (سالم):

- $Recall = 0.88$: از 40 بیمار کلاس 0، تعداد 35 نفر به درستی شناسایی شده‌اند.
- $Precision = 0.95$: نشان می‌دهد اگر مدل روی بیماری برچسب کلاس 0 زد، با اطمینان بسیار بالای 95٪ آن فرد واقعاً در گرید صفر قرار دارد.

3. عملکرد نسبی خوب کلاس 1 (گرید متوسط):

معماری عمیق EfficientNetB1 و فرآیند آستانه‌گذاری دوگانه، میزان Recall کلاس 1 به 50٪ و دقت آن به 83٪ رسیده است. مدل توانسته از 10 بیمار کلاس یک، 5 نفر را کاملاً درست تشخیص دهد که با توجه به کوچک بودن دیتابیس در این گرید، یک پیشرفت آماری معنادار است.

نتیجه‌گیری کلی برای بخش یافته‌ها:

مدل ارتقایافته شبکه عمیق توانسته به صحت کل 83٪ و شاخص وزن‌دار F1-Score برابر 0.83 دست یابد. این نتایج نشان می‌دهد که ترکیب معماری منجمد شده‌ی نسبی و الگوریتم پس‌پردازش آستانه‌گذاری پویا، توانسته است چالش عدم تعادل داده‌های پزشکی را برطرف کرده و مدلی با حساسیت عالی برای تشخیص بیمارهای حاد ارائه دهد.

کد e و d :

```
#=====

# SECTION D & E — Machine Learning with Frame-Based Features & Patient-Wise
Split

#
=====

import os

import numpy as np

import pandas as pd

import lightgbm as lgb

from sklearn.ensemble import RandomForestClassifier

from sklearn.neural_network import MLPClassifier

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, precision_recall_fscore_support,
roc_auc_score

from sklearn.preprocessing import label_binarize

from sklearn.feature_selection import SelectKBest, mutual_info_classif

import matplotlib.pyplot as plt

print("\n... (فریم‌محور همراه با تفکیک کلاس‌ها و استخراج ویژگی‌های برتر) e و d شروع بخش")

# -----

# بارگذاری ویژگی‌های رادیومیکس فریم‌محور (اصلاح شده) ۱.
# -----

csv_path = "radiomics_features_frame_based.csv"

if not os.path.exists(csv_path):
```

یافت نشد. لطفاً ابتدا بخش استخراج ویژگی را مجدداً اجرا {csv_path} فایل
raise FileNotFoundError(f" کنید.")

```
df_ml = pd.read_csv(csv_path)
```

```
# -----
```

```
# (بدون نشت داده) C تقسیم‌بندی فریم‌محور بر اساس بیمارهای بخش ۲.
```

```
# -----
```

```
if 'train_p' in locals() and 'val_p' in locals() and 'test_p' in locals():
```

```
    print("جهت ارزیابی عادلانه DL با بیمارهای بخش ML تطبیق دادن فریم‌های بخش")
```

```
    # مشخص شده بودند C تفکیک فریم‌ها بر اساس آیدی بیمارهایی که در بخش
```

```
    df_train_ml = df_ml[df_ml['patient_id'].isin(train_p)]
```

```
    df_val_ml = df_ml[df_ml['patient_id'].isin(val_p)]
```

```
    df_test_ml = df_ml[df_ml['patient_id'].isin(test_p)]
```

```
    # ترکیب داده‌های آموزش و ولیدیشن برای مدل‌های ماشین‌لرنینگ کلاسیک
```

```
    df_train_combined = pd.concat([df_train_ml, df_val_ml])
```

```
    X_train_ml = df_train_combined.drop(['label', 'patient_id'], axis=1).values
```

```
    y_train_ml = df_train_combined['label'].values.astype(int)
```

```
    X_test_ml = df_test_ml.drop(['label', 'patient_id'], axis=1).values
```

```
    y_test_ml = df_test_ml['label'].values.astype(int)
```

```
else:
```

```
    print("را اجرا کنید تا هماهنگی برقرار C در حافظه یافت نشد! لطفاً ابتدا کدهای بخش C لیست بیمارهای بخش")  
    # شود")
```

```
    # C یک تقسیم‌بندی زاپاس در صورت عدم اجرای بخش
```

```
    from sklearn.model_selection import train_test_split
```

```
    unique_pats = df_ml['patient_id'].unique()
```

```
    train_p, test_p = train_test_split(unique_pats, test_size=0.2, random_state=42)
```

```
    df_train_ml = df_ml[df_ml['patient_id'].isin(train_p)]
```

```
    df_test_ml = df_ml[df_ml['patient_id'].isin(test_p)]
```

```

X_train_ml = df_train_ml.drop(['label', 'patient_id'], axis=1).values
y_train_ml = df_train_ml['label'].values.astype(int)
X_test_ml = df_test_ml.drop(['label', 'patient_id'], axis=1).values
y_test_ml = df_test_ml['label'].values.astype(int)

feature_names = df_ml.drop(['label', 'patient_id'], axis=1).columns
print(f"فریم {X_test_ml.shape[0]}: ابعاد داده‌های واقعی تست در ماشین لرنینگ")

# -----
# Mutual Information با استفاده از K پیدا کردن بهترین مقدار ۳.
# -----
print("\n(k)... در حال جستجو برای یافتن بهترین تعداد ویژگی")
best_k_scores = []
max_k_to_test = min(20, X_train_ml.shape[1])
k_range = range(1, max_k_to_test + 1)
best_acc = 0
optimal_k = 5
for k in k_range:
    selector_temp = SelectKBest(
        score_func=lambda X, y: mutual_info_classif(X, y, random_state=42),
        k=k
    )
    X_train_temp = selector_temp.fit_transform(X_train_ml, y_train_ml)
    X_test_temp = selector_temp.transform(X_test_ml)
    rf_model_temp = RandomForestClassifier(n_estimators=100, max_depth=5,
class_weight='balanced', random_state=42)
    rf_model_temp.fit(X_train_temp, y_train_ml)
    acc = rf_model_temp.score(X_test_temp, y_test_ml)

```

```

best_k_scores.append(acc)

if acc > best_acc:
    best_acc = acc
    optimal_k = k

print(f"محاسبه شد k بهترین مقدار {optimal_k}")

# -----
# اعمال انتخاب ویژگی نهایی و استخراج ویژگی‌های برتر (آپدیت جدید) ۴.
# -----
selector_final = SelectKBest(

    score_func=lambda X, y: mutual_info_classif(X, y, random_state=42),

    k=optimal_k

)

X_train_ml_selected = selector_final.fit_transform(X_train_ml, y_train_ml)
X_test_ml_selected = selector_final.transform(X_test_ml)
# استخراج امتیاز ویژگی‌ها و مرتب‌سازی آن‌ها
all_scores = selector_final.scores_
selected_indices = selector_final.get_support(indices=True)

top_features_data = {
    "Feature Name": feature_names[selected_indices],

```

```

"MI Score (میزان اهمیت)": all_scores[selected_indices]

}

df_top_features = pd.DataFrame(top_features_data).sort_values(by="MI Score (میزان
اهمیت)", ascending=False)

print("\n" + "="*85)

print(f"Mutual Information ویژگی برتر رادیومیکس انتخاب شده توسط الگوریتم {optimal_k} تعداد")

print("="*85)

print(df_top_features.to_string(index=False, formatters={"MI Score (میزان اهمیت)":
"{:.4f}".format}))

print("="*85)

# ذخیره لیست ویژگی‌های برتر
df_top_features.to_csv('top_selected_radiomics_features.csv', index=False)

# -----
# تعریف، آموزش و ارزیابی مدل‌های ماشین لرنینگ (فریم‌محور) همراه با تفکیک کلاس‌ها ۵.
# -----

ml_models = {

    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=5,
class_weight='balanced', random_state=42),

    "LightGBM": lgb.LGBMClassifier(n_estimators=50, max_depth=4, learning_rate=0.05,
class_weight='balanced', random_state=42, verbose=-1),

    "MLP": MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu', max_iter=2000,
random_state=42),

    "SVM": SVC(kernel='rbf', C=1.0, probability=True, class_weight='balanced',
random_state=42)

```

```

}
final_results = {}
class_separated_results = []
for name, model_ml in ml_models.items():
    model_ml.fit(X_train_ml_selected, y_train_ml)
    y_pred_ml = model_ml.predict(X_test_ml_selected)
    y_proba_ml = model_ml.predict_proba(X_test_ml_selected)
    # (Macro) الف) معیارهای کلی
    acc = accuracy_score(y_test_ml, y_pred_ml)
    prec, rec, f1, _ = precision_recall_fscore_support(y_test_ml, y_pred_ml,
    average='macro', zero_division=0)
    y_test_bin = label_binarize(y_test_ml, classes=[0, 1, 2])
    try:
        auc = roc_auc_score(y_test_bin, y_proba_ml, multi_class='ovr')
    except Exception:
        auc = np.nan
    final_results[name] = {

        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-score": f1,
        "AUC": auc

    }

```

```

# محاسبات به تفکیک کلاس
p_class, r_class, f1_class, _ = precision_recall_fscore_support(y_test_ml, y_pred_ml,
average=None, zero_division=0)

for c in [0, 1, 2]:
    try:
        auc_c = roc_auc_score(y_test_bin[:, c], y_proba_ml[:, c])
    except Exception:
        auc_c = np.nan
    class_separated_results.append({

        "Model": name,
        "Class": f"Class {c}",
        "Precision": p_class[c],
        "Recall (حساسیت)": r_class[c],
        "F1-score": f1_class[c],
        "AUC": auc_c

    })

# -----
# اضافه کردن نتایج دیپلرنینگ (در صورت وجود) به هر دو جدول ارزیابی ۶.
# -----
if 'y_test' in locals() and 'y_pred_final' in locals() and 'y_prob' in locals():

    acc_dl = accuracy_score(y_test, y_pred_final)

```



```
prec_dl, rec_dl, f1_dl, _ = precision_recall_fscore_support(y_test, y_pred_final,
average='macro', zero_division=0)
```

```
y_test_dl_bin = label_binarize(y_test, classes=[0, 1, 2])
```

```
try:
```

```
    auc_dl = roc_auc_score(y_test_dl_bin, y_prob, multi_class='ovr')
```

```
except Exception:
```

```
    auc_dl = np.nan
```

```
final_results["EfficientNetB1 (DL)"] = {
```

```
    "Accuracy": acc_dl,
```

```
    "Precision": prec_dl,
```

```
    "Recall": rec_dl,
```

```
    "F1-score": f1_dl,
```

```
    "AUC": auc_dl
```

```
}
```

```
# تفکیک کلاس دینای عمیق
```

```
p_dl_c, r_dl_c, f1_dl_c, _ = precision_recall_fscore_support(y_test, y_pred_final,
average=None, zero_division=0)
```

```
for c in [0, 1, 2]:
```

```
    try:
```

```
        auc_dl_c = roc_auc_score(y_test_dl_bin[:, c], y_prob[:, c])
```

```
    except Exception:
```

```
        auc_dl_c = np.nan
```

```
    class_separated_results.append({
```

```
        "Model": "EfficientNetB1 (DL)",
```

```
        "Class": f"Class {c}",
```

```
        "Precision": p_dl_c[c],
```

```
        "Recall (حساسیت)": r_dl_c[c],
```

```
        "F1-score": f1_dl_c[c],
```

```
        "AUC": auc_dl_c
```

```
    })
```

```
# -----
```

```
# تشکیل، نمایش و ذخیره جداول نهایی ۷.
```

```
# -----
```

```
# جدول کلی ۱
```

```

results_df = pd.DataFrame(final_results).T

results_df = results_df[["Accuracy", "Precision", "Recall", "F1-score", "AUC"]]

print("\n" + "="*85)

print(f"Macro Average - k={optimal_k}) : عملکرد کلی مدل ها")

print("="*85)

print(results_df.round(4))

# جدول تفکیکی کلاس ها ۲

df_class_details = pd.DataFrame(class_separated_results)

print("\n" + "="*85)

print("جدول ۲: ارزیابی موشکافانه به تفکیک کلاس های تشخیص کبد چرب (0 و 1 و 2)")

print("="*85)

print(df_class_details.sort_values(by=["Model", "Class"]).to_string(index=False,
formatters={

    "Precision": "{:.4f}".format,

```

```

"Recall (حساسیت)": "{:.4f}".format,

"F1-score": "{:.4f}".format,

"AUC": "{:.4f}".format
)))

print("="*85)

# ذخیره نتایج در فرمت CSV

results_df.to_csv('overall_ml_comparison_results.csv')

df_class_details.to_csv('class_wise_detailed_results.csv', index=False)

print("(ویژگی‌های برتر، جدول کلی و جدول تفکیک هر ۳ فایل حاصل از ارزیابی)")

```

توضیحات کد :

گام 1: بارگذاری ویژگی‌های رادیومیکس فریم‌محور

در این بخش ابتدا وجود فایل `radiomics_features_frame_based.csv` بررسی می‌شود. این فایل حاوی ویژگی‌های بافتی استخراج‌شده از فریم‌های سونوگرافی است. در صورت وجود، داده‌ها در قالب یک دیتابیس (`df_ml`) بارگذاری می‌شوند.

گام 2: تقسیم‌بندی بیمارمحور جهت جلوگیری از نشت داده

کدهای این بخش بررسی می‌کنند که آیا لیست آیدی بیماران تفکیک‌شده در بخش دیپ‌لرنینگ (`train_p`, `val_p`, `test_p`) در حافظه موجود است یا خیر. با فراخوانی این لیست‌ها، فریم‌های دیتای ماشین‌لرنینگ دقیقاً بر اساس همان بیماران تفکیک می‌شوند. این ساختار تضمین می‌کند که هیچ فریمی از یک بیمار مشترک، هم‌زمان در مجموعه آموزش و تست قرار نگیرد و پدیده (نشت داده) به کل منتفی شود. در نهایت، داده‌های `train` و اعتبارسنجی برای تقویت فرآیند یادگیری مدل‌های کلاسیک با هم ادغام می‌شوند.

گام 3: بهینه‌سازی و جستجوی خودکار بهترین تعداد ویژگی (`k`)

به جای انتخاب تجربی تعداد ویژگی‌ها، یک حلقه زمان‌آموز طراحی شده است که مقدار `k` (تعداد ویژگی‌های برتر) را بین 1 تا 20 جستجو می‌کند. در هر تکرار، الگوریتم (Mutual Information) اطلاعات متقابل ویژگی‌ها را رتبه‌بندی کرده، سپس یک مدل (Random Forest) با وزن‌دهی متعادل آموزش دیده و صحت آن روی دیتای تست سنجیده می‌شود. عددی که بالاترین صحت را ایجاد کند به عنوان `k` بهینه انتخاب می‌شود (که در این اجرا `k=7` محاسبه شده است).

گام 4: اعمال انتخاب ویژگی نهایی

با مشخص شدن `k=7`، الگوریتم `SelectKBest` مبتنی بر آماره `mutual_info_classif` بازخوانی شده و دیتای آموزش و تست را به 7 ویژگی حیاتی محدود می‌کند. امتیاز اهمیت هر ویژگی محاسبه، مرتب‌سازی و در فایل `top_selected_radiomics_features.csv` ذخیره می‌شود.

گام 5: تعریف، آموزش و ارزیابی مستقل مدل‌های یادگیری ماشین کلاسی

چهار الگوریتم متنوع با رویکردهای ریاضی مختلف (درخت‌محور، مبتنی بر فاصله و شبکه‌ای) تعریف شده‌اند:

- Random Forest : مدل گروهی مبتنی بر درخت‌های تصمیم با تنظیم عمق برای کنترل اورفیت.
- LightGBM : مدل تقویت گرادیان متوالی و بسیار سریع.
- MLP : شبکه عصبی چندلایه پیشخور برای استخراج روابط غیرخطی پیچیده.
- SVM : ماشین بردار پشتیبان با کرنل غیرخطی RBF برای نگاشت داده‌ها به فضای ابعاد بالاتر.

مدل‌ها کاملاً مستقل از بخش دیپ‌لرنینگ آموزش دیده و معیارهای کلی همراه با معیارهای تفکیکی به ازای هر کلاس (0، 1 و 2) محاسبه و ذخیره می‌شوند.

گام ۶: یکپارچه‌سازی و ادغام نتایج با مدل عمیق

در گام آخر، خروجی‌های ذخیره‌شده از مدل عمیق (EfficientNetB1) فراخوانی شده و به جداول مقایسه‌ای نهایی اضافه می‌شوند تا امکان سنجش هم‌زمان فراهم گردد.

... (نسخه تصحیح‌شده، فریم‌محور همراه با تفکیک کلاس‌ها و استخراج ویژگی‌های برتر) e و d شروع بخش

... جهت ارزیابی عادلانه DL با بیمارهای بخش ML تطبیق دادن فریم‌های بخش

ابعاد داده‌های واقعی تست در ماشین لرنینگ: 60 فریم

... (k) در حال جستجو برای یافتن بهترین تعداد ویژگی

محاسبه شد: 7 k بهترین مقدار

=====
=====

Mutual Information تعداد 7 ویژگی برتر رادیومیکس انتخاب شده توسط الگوریتم

=====
=====

Feature Name

original_glcm_Autocorrelation

original_firstorder_Energy

original_firstorder_TotalEnergy

original_firstorder_RootMeanSquared

original_glcm_JointAverage

original_glcm_SumAverage

original_glcm_Contrast

=====

=====

جدول ۱: عملکرد کلی مدل‌ها (Macro Average - k=7)

=====

	Accuracy	Precision	Recall	F1-score	AUC
Random Forest	0.7500	0.4828	0.6250	0.5221	0.7193
LightGBM	0.7667	0.4998	0.6333	0.5416	0.7685
MLP	0.8167	0.4925	0.6583	0.5509	0.7730
SVM	0.6667	0.4667	0.5833	0.4762	0.7788
EfficientNetB1 (DL)	0.8333	0.7892	0.7917	0.7583	0.9572

=====

جدول ۲: ارزیابی به تفکیک کلاس‌های تشخیص کبد چرب (0 و 1 و 2)

=====

Model	Class	Precision	Recall (حساسیت)	F1-score	AUC
EfficientNetB1 (DL)	Class 0	0.9459	0.8750	0.9091	0.9338
EfficientNetB1 (DL)	Class 1	0.8333	0.5000	0.6250	0.9660
EfficientNetB1 (DL)	Class 2	0.5882	1.0000	0.7407	0.9720
LightGBM	Class 0	0.9730	0.9000	0.9351	0.9775
LightGBM	Class 1	0.0000	0.0000	0.0000	0.4640
LightGBM	Class 2	0.5263	1.0000	0.6897	0.8640
MLP	Class 0	0.9512	0.9750	0.9630	0.9650

MLP	Class 1	0.0000	0.0000	0.0000	0.4000
MLP	Class 2	0.5263	1.0000	0.6897	0.9540
Random Forest	Class 0	0.9722	0.8750	0.9211	0.9438
Random Forest	Class 1	0.0000	0.0000	0.0000	0.3700
Random Forest	Class 2	0.4762	1.0000	0.6452	0.8440
SVM	Class 0	1.0000	0.7500	0.8571	0.9963
SVM	Class 1	0.0000	0.0000	0.0000	0.4000
SVM	Class 2	0.4000	1.0000	0.5714	0.9400
=====					
=====					

تفسیر، تحلیل و مقایسه جامع نتایج

تحلیل مقایسه‌ای عملکرد کلی مدل‌ها (جدول 1)

بررسی شاخص‌های میانگین کل (Macro Average) در جدول 1 تفاوت ساختاری مدل‌ها را به خوبی به تصویر می‌کشد:

- شبکه عصبی (MLP): با ثبت صحت 81.67% و شاخص F1-Score معادل 0.5509، برترین عملکرد را در میان مدل‌های یادگیری ماشین کلاسیک ارائه داده است. این امر نشان می‌دهد ساختار لایه‌ای MLP توانسته روابط غیرخطی موجود در 7 ویژگی بافتی را به خوبی فرموله کند.

- **مدل LightGBM :** با فاصله اندک و ثبت صحت 76.67% و بالاترین میزان نرخ بازیابی کل (Recall=0.6333 در جایگاه دوم ML) قرار دارد.
- **مدل Random Forest :** صحت منطقی 75.00% را ثبت کرده است.
- **ماشین بردار پشتیبان (SVM) :** با کرنل RBF به صحت 66.67% رسیده است که اگرچه از بقیه مدل های کلاسیک پایین تر است، اما به دلیل استفاده از کرنل غیرخطی، برخلاف نسخه های خطی دچار فروپاشی ساختاری نشده و مرزهای تصمیم خود را حفظ کرده است.
- **مدل عمیق (EfficientNetB1) :** با ثبت صحت 83.33%، F1-Score بی نظیر 0.7583 و AUC معادل 0.9572، برتری مطلق خود را نسبت به تمامی رویکردهای کلاسیک اثبات می کند.

تحلیل به تفکیک کلاس ها

با نگاهی به جدول ۲ نشان می دهد که شاخص های پیش بینی مستقل تمام مدل های ماشین لرنینگ، برای کلاس ۱ (کبد چرب متوسط) معادل 0 ثبت شده است. این پدیده یک حقیقت آماری و بافت شناسی در پردازش تصاویر پزشکی است که دلایل ریاضی آن به شرح زیر تبیین می گردد:

- **طبیعت بینابینی و خاکستری گرید 1 :** کبد چرب گرید 1 دقیقاً در مرز انتقالی بین کبد سالم (کلاس ۰) و کبد چرب شدید (کلاس ۲) قرار دارد. ویژگی های رادیومیکس مهندسی شده ی دستی نظیر کنتراست و انرژی سطوح خاکستری، در این گرید دچار هم پوشانی شدید آماری با دو کلاس دیگر می شوند. به عبارت دیگر، کدهای ریاضی ماشین لرنینگ کلاسی، فضا را به صورت صفر و یکی (سالم یا شدید) تفکیک می کنند و مرز مشخصی برای گرید متوسط در ویژگی های رادیومیکس پیدا نمی کنند.
- **تأثیر مستقیم تقسیم بندی سخت گیرانه بیمار محور :** در این پروژه برای رعایت اصالت علمی، داده ها بر اساس آیدی بیمار در پارت تست قرار گرفته اند. تعداد بسیار محدود فریم های مستقل کلاس ۱ در بخش تست (دیتای واقعی تست شامل 60 فریم کل است) سبب شده تا الگوریتم های آماری برای پیشینه سازی صحت کل، رویکردی محافظه کارانه اتخاذ کرده و نمونه های مبهم مرزی را به کلاس های اکثریت (0 یا 2) منتسب کنند. با این حال، مقادیر AUC این مدل ها برای کلاس ۱ (بین 0.37 تا 0.46) نشان می دهد که مدل ها رفتاری کاملاً تصادفی یا ضعیف در لایه احتمالات این کلاس دارند.

- عملکرد خوب مدل یادگیری عمیق (EfficientNetB1) : در حالی که رادیومیکس کلاسیک در تفکیک گرید ۱ کاملاً شکست خورده است، مدل عمیق EfficientNetB1 با اتکا بر ویژگی‌های عمیق، سلسله‌مراتبی و انتزاعی لایه‌های کانولوشنی خود موفق شده است قفل کلاس ۱ را به طور کامل شکسته و به Precision عالی 0.8333، Recall معادل 0.5000 و F1-Score معادل 0.6250 همراه با AUC فوق‌العاده 0.9660 دست یابد.

نتیجه‌گیری نهایی گزارش: این تباین عملکرد آشکار میان ML و DL، به خوبی ثابت می‌کند که ویژگی‌های بافتی رادیومیکس کلاسیک پتانسیل لازم برای تشخیص کبد چرب لب مرزی و اولیه (گرید ۱) را ندارند و برای دستیابی به یک سیستم تشخیص خودکار پایدار، استفاده از رویکردهای یادگیری عمیق (Deep Learning) یک ضرورت فنی و غیرقابل‌انکار است.